

Automated Compilation Including Dropouts: Tolerating Defective Components in Stabiliser Codes

Stasiu Wolanski *

Riverlane, St Andrew's House, Cambridge, UK

Dept. of Physics and Astronomy, University College London, UK

November 2025

Abstract

Utility-scale solid-state quantum devices will need to fabricate quantum devices at scale using imperfect processes. By introducing tolerance to fabrication defects into the design of the quantum devices, we can improve the yield of usable quantum chips and lower the cost of useful systems. Automated Compilation Including Dropouts (ACID) is a framework that works in the ancilla-free (or ‘middle-out’) paradigm, to generate syndrome extraction circuits for general stabiliser codes in the presence of defective couplers or qubits. In the ancilla-free paradigm, we do not designate particular qubits as measurement ancillas, instead measuring stabilisers using any of the data qubits in their support. This approach leads to a great deal of flexibility in how syndrome extraction circuits can be implemented. Given a stabiliser code and information about the functional and defective couplers between the qubits, ACID works by constructing and solving an optimisation problem within the ancilla-free paradigm to find a short syndrome extraction circuit. Applied to the surface code, ACID produces syndrome extraction circuits of depth between $1\times$ (no overhead) and $1.5\times$ relative to the depth of defect-free circuits. The original LUCI algorithm, the best prior art, yielded a $2\times$ overhead, so ACID offers a significant time saving. The yield of surface code chips with a logical error rate at most $10\times$ the dropout-free baseline is up to $3\times$ higher using ACID than using original LUCI. I demonstrate the broad applicability of ACID by compiling syndrome extraction circuits for bivariate bicycle codes and the colour code. For these circuits, we incur a circuit-depth overhead of between $1\times$ (no overhead) and $2.5\times$ relative to defect-free circuits. Depending on the underlying connectivity, we can often accommodate isolated defective couplers and qubits with only an order-unity impact on the logical error rate. I believe this work is the first to simulate both of these families of codes in the presence of fabrication defects.

1 Introduction

To solve problems of broad scientific and commercial interest, quantum computers will need to maintain the integrity of the stored logical information for a duration billions or trillions of times longer than the decoherence time of their most fundamental components. In particular, for the artificial qubits (superconducting and spin) which are the main target of this paper, fabrication and control techniques must be improved to bring down the decoherence probabilities associated with each operation. To help bridge the gap between current physical error rates and the logical error rates we need, we can exponentially amplify the benefit of such engineering advances at the cost of introducing many additional physical qubits, by encoding our information redundantly in a quantum error-correcting code.

Since these quantum error-correcting codes were created to handle imperfections in circuit execution, it would seem natural that they could also handle known, permanent imperfections in the quantum devices themselves. Even state-of-the-art fabrication techniques do not have a perfect yield [1, 2], and requiring that each quantum chip is perfect—that every qubit and every coupler between the qubits is functional and high fidelity—would incur a substantial, perhaps unviable, manufacturing overhead in post-selecting for chips that turned out perfectly. I

*stasiu.wolanski@riverlane.com

note that the classical semiconductor industry, whose products are far less delicate devices than transmons or SETs, routinely builds redundancy into its chip architectures to accommodate inevitable defects. In the same way that QECCs amplify improvements in gate execution, some level of defect-tolerance in the architecture of QPUs could substantially amplify the gain in device yield afforded by improvements in fabrication.

I work with the assumption that certain qubits and couplers on our device, known at the time of circuit compilation, are completely non-functional. In the case of broken (‘dropped’) couplers, this means that the two qubits the coupler connects can no longer interact with one another, whereas ‘dropped’ qubits cannot interact with any of their neighbours; dropping a qubit is equivalent to dropping all its connected couplers. It seems likely similar techniques might be useful where a qubit is functional but is known to be highly error-prone.

There is plenty of literature proposing techniques to handle dropouts (as I will call them henceforth) in the surface code [3–7] (see also [8], where dropouts on Floquet codes are explored). The state-of-the-art is the LUCI framework [9, 10], which can handle multiple dropped couplers and qubits, often without losing code distance. However, LUCI syndrome extraction circuits have a circuit depth twice that in the dropout-free case. This results in an overall increase in computation time by at least the same factor, and in addition may increase logical error rates as each round provides a longer window for errors to accrue.

In this work, I introduce Automated Compilation Including Dropouts (ACID), a framework within which any CSS stabiliser code can be compiled to a syndrome extraction cycle that accommodates any pattern of dropouts, with any suitable underlying qubit connectivity. ACID generalises LUCI, treating general codes in the ancilla-free (or ‘middle-out’) paradigm. It then encodes the problem of finding a syndrome extraction circuit into an integer linear program (ILP) and optimises using standard techniques to find a short syndrome extraction circuit. I demonstrate compilation of the surface code, the bivariate bicycle codes and the colour code, each using multiple underlying qubit connectivities. A full characterisation of performance of the nine code-connectivity setups for up to three dropped qubits and three dropped couplers is given in Appendix D.

I demonstrate ACID applied to the bivariate bicycle code assuming both the degree-5 connectivity introduced in [11], and an alternative degree-6 connectivity that introduces more redundancy and leads to greater robustness in qubit dropouts. These result in syndrome extraction circuits of depth between $1\times$ and $2.5\times$ the dropout-free case, depending on the number of dropouts. I applied ACID to both the ‘middle-out’ and ‘superdense’ formulations of the colour code presented in [12], trying the former on both a hex-grid and square-grid lattice and the latter on a square-grid lattice. Again, the syndrome extraction circuits are of length between $1\times$ and $2.5\times$ the dropout-free circuits, with the three configurations each offering a different balance of qubit overhead, syndrome extraction circuit depth, and logical error rate.

Applied to the surface code, ACID maintains code-capacity distance in the same situations that LUCI does, and can typically find syndrome extraction cycles that have either the same depth or $1.5\times$ the depth of the dropout-free circuits—a reduction in time-overhead of between 25% and 50%. Its distance-preserving properties are comparable: depending on whether we assume an underlying square-grid (degree-4) or hex-grid (degree-3) connectivity, we can tolerate the loss of certain qubits and couplers without losing distance. I implemented a version of the LUCI algorithm as a special case of ACID with additional constraints (Appendix E), and find that ACID is able to maintain error rates within an order of magnitude of the dropout-free baseline up to $3\times$ more often than LUCI for small numbers of dropped qubits and couplers (Figure 1).

The source code for ACID, with examples, is available on GitHub [13]. The repository also hosts visualisations of syndrome extraction circuits produced by the algorithm.

Update (January 2026): the comparison made here is to a reproduction of LUCI insofar as it is detailed in the original two works [9, 10]. Shortly after the release of this manuscript, Anker and Debroy [14] showed by means of a direct comparison that ACID produces surface code syndrome extraction circuits with higher error rates than those produced by the closed-source implementation of LUCI upon which the original LUCI papers were based, explaining that their implementation contains heuristic optimisations that are not presented in the original work. Anker and Debroy also detail a new version of LUCI using very similar techniques to those in the present work, but equipped with a set of additional, explicitly described optimisation heuristics, meaning the most up-to-date version of the LUCI algorithm is both more performant and more reproducible than previous versions, and is also capable of matching ACID’s time overhead. In my view, the newest version of LUCI should be considered the state-of-the-art for tolerating fabrication defects in the surface code. Promisingly, the heuristics presented therein are compatible with the general framework of ACID, and future work should merge the two with the aim of producing similarly optimised syndrome extraction circuits for other stabiliser codes.

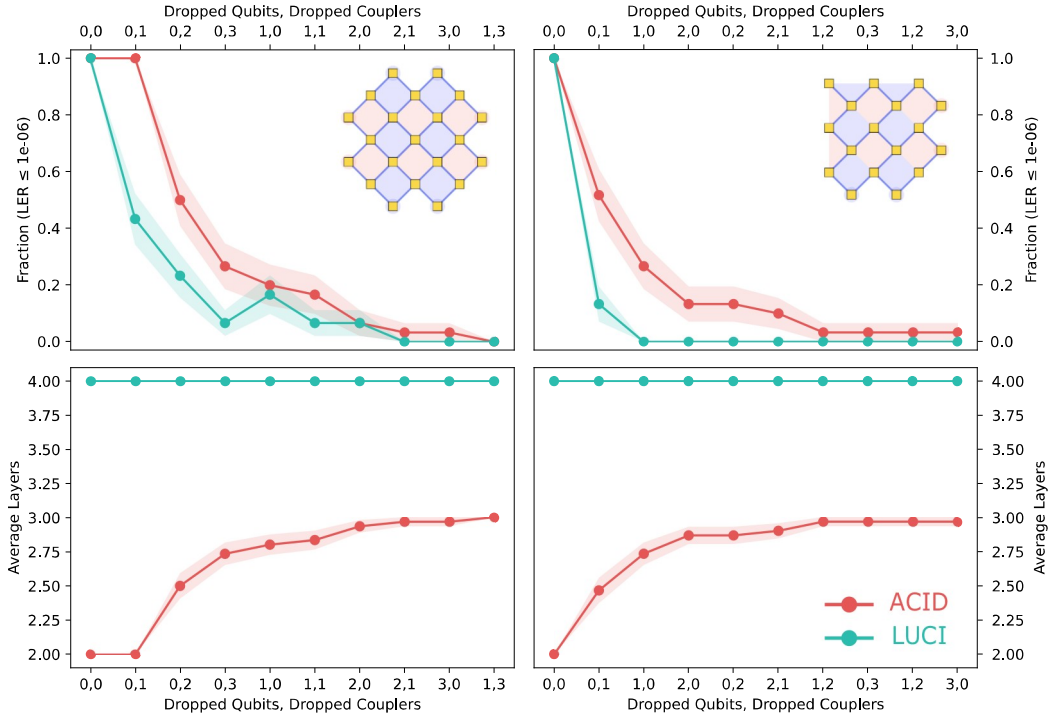


Figure 1: A comparison between ACID and the state-of-the-art LUCI algorithm [9] when both are applied to the distance-11 surface code. Top left: the proportion of cases where ACID and LUCI are able to produce syndrome extraction circuits with a logical error rate less than 10^{-6} for a distance-11 surface code implemented on a device with square-grid connectivity, assuming a random subset of qubits and couplers are defective. The numbers of defective qubits and defective couplers are shown on the x -axis, and the shaded region is the standard binomial error. Bottom left: In the same situations, the average number of ‘contraction layers’ (Section 2) required to construct a round of syndrome extraction, plotted with the standard error in the mean. LUCI always requires 4 rounds, whereas ACID is observed to never require more than 3. Top right and bottom right: The same comparison for the distance-11 surface code implemented on a device with hex-grid connectivity. These plots are a summary of the full data provided in Appendix D.

2 Ancilla-free QEC

Vasmer and Kubica introduced the ‘morphing’ technique for stabiliser codes, in which a subset of the physical qubits in a stabiliser code are disentangled from the rest using a unitary circuit, leaving behind a different code that contains the same logical information but has fewer physical qubits [15] and different error-correction performance. The ancilla-free (or ‘middle-out’) view of QEC was first proposed in [16] for the surface code, generalised to the colour code in [12] and generalised to the bivariate bicycle codes in [11]. All three works can be seen as an application of the original morphing technique. In the ancilla-free framework, we map a Calderbank-Shor-Steane (CSS) stabiliser code with n qubits onto n physical qubits, without any additional qubits designated for measuring the stabilisers. In other words, stabilisers are defined as subsets of the physical qubits along with a Pauli type (X/Z).

The ancilla-free framework in general requires the same number of physical qubits as in ancilla-based QEC to implement codes with the same distance and logical qubits, but has the key advantage that it allows far more flexibility in constructing syndrome extraction circuits. In [16] and [11] this flexibility is leveraged to reduce the connectivity required to implement surface and bivariate bicycle codes. In the LUCI framework [9, 10] and in the present work, this flexibility allows us to work around dropouts (and optionally maintain the reduced connectivity requirements).

One way to understand the ancilla-free paradigm is by considering the evolution of a set of privileged generators of the instantaneous stabiliser group (ISG) throughout a syndrome extraction round. At the start of a round our qubits are stabilised by the stabilisers of the input code, and we take these as our privileged generators.

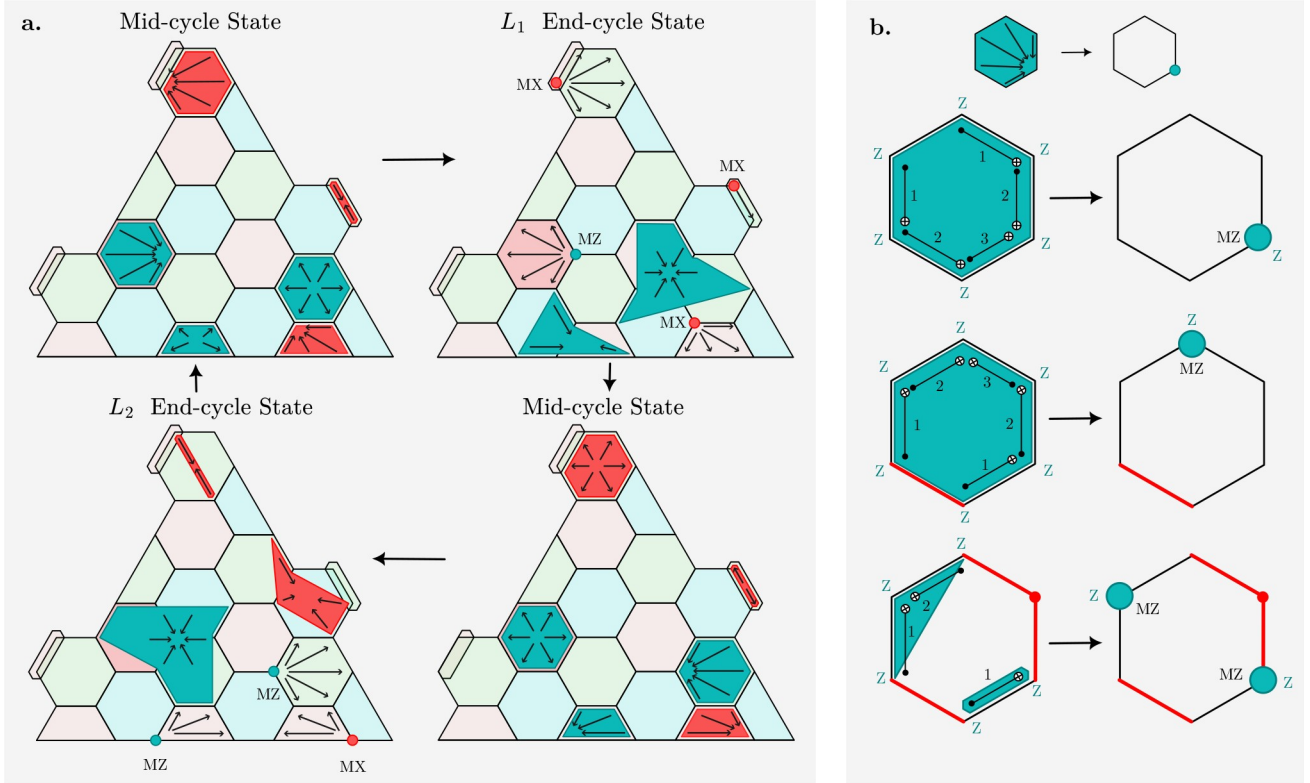


Figure 2: *The ancilla-free approach to implementing quantum error-correcting codes.* **a.** A single round of one possible syndrome extraction cycle for the triangular colour code, that consists of two contraction layers. I've highlighted three X -type (red) and three Z -type (teal) stabilisers and track the corresponding instantaneous stabilisers through the syndrome extraction cycle. We start (top-left) the cycle in the mid-cycle state, which comes from the standard definition of the colour code, and contains no ancilla qubits. In the first round, the three X stabilisers and one of the Z stabilisers are contracted, and the remaining two Z stabilisers are expanded. We perform three timesteps' worth of CNOT gates to take us to the second stage, known as the L_1 end-cycle state. The contracted stabilisers are now each localised onto a single qubit which we measure in the appropriate basis. We undo the CNOTs to return to the mid-cycle state. In the second layer, we contract a different subset of the stabilisers (producing the L_2 end-cycle state), such that each stabiliser is measured at least once over the two rounds. Note that the X stabiliser in the bottom-right is measured in both layers, but is contracted onto a different qubit in each.

b. Here I show how a weight-six stabiliser with a cyclic local connectivity is affected by the presence of dropouts. In the top case, we have no dropouts and we arbitrarily choose a contraction schedule onto one of the qubits, subject only to compatibility with the schedules of neighbouring stabilisers. In the middle case, the presence of a dropped-out coupler forces us to use a different contraction schedule (note there are still two contraction schedules to choose from). In the bottom case, the presence of both a defective coupler and a defective qubit causes the remaining qubits in the stabiliser to become disconnected. In this case, we define 'quasi-stabilisers' from the remaining connected components and define product stabilisers and gauge qubits from them, forming a subsystem code. We then measure these stabilisers separately, and classically multiply the outcomes of multiple quasi-stabilisers to form deterministic detectors.

Confusingly, the input code is called the ‘mid-cycle code’ and the qubits are said to be in the ‘mid-cycle state’ at this point in the circuit.

To measure some stabiliser S of the mid-cycle code, we can perform a Clifford operation C , and then measure the single-qubit Pauli operator P , where $C^{-1}SC = P$. The process of localising the support of a stabiliser generator onto a single qubit is called ‘contracting’. Furthermore, we can measure a subset of m stabiliser elements in parallel if $\{S_1, S_2, \dots, S_m\}$ satisfy $C^{-1}S_jC = P_j$ for some set of single qubit Pauli operators $\{P_1, P_2, \dots, P_m\}$. These Cliffords must also be broken down into CNOT gates that respect device connectivity. We refer to these joint measurement operations as ‘contraction layers’.

These are depicted in Figure 2a. In the first half of each contraction layer, we choose some compatible subset of the stabilisers to measure, and perform CNOTs to perform the Clifford C . This has the effect of contracting the support of each stabiliser so that it consists of a single qubit, which is called the ‘root qubit’. This root qubit is then measured and reset in the appropriate basis. The stabilisers that are not being contracted in a given round have their support modified in some complex way, according to $\bar{S} \rightarrow C^{-1}\bar{S}C$. The form of these expanded stabilisers halfway through a syndrome extraction layer, given by $\{C^{-1}\bar{S}_iC\}$, defines the ‘end-cycle code’, which contains the same logical information as the mid-cycle code, but uses fewer than n qubits and in general has a reduced distance. In the second half of each contraction layer, we perform the same CNOTs in reverse to enact C^{-1} , ‘un-contracting’ the contracted stabilisers from their singular support on the root qubits and ‘un-expanding’ the expanded stabilisers so all the stabiliser generators return to their original ‘mid-cycle’ state.

In the next contraction layer, we choose a different (not necessarily disjoint) set of stabilisers to contract, and repeat the process of contracting these onto some chosen root qubits, measuring and resetting the root qubits, and undoing the contraction. We perform as many of these layers as we need to measure each stabiliser at least once. Note that in some cases, the ancilla-free and ancilla-based paradigms can describe the same syndrome extraction circuit: notably, in the most basic distance- d rotated surface code circuits presented in [16], the mid-cycle code has $\sim 2d^2$ physical qubits, and each end-cycle code can be viewed as an unrotated surface code supported on $\sim d^2$ physical ‘data’ qubits alongside $\sim d^2$ disentangled ‘ancilla’ qubits. In this work, we privilege the mid-cycle code, fixing it and allowing the end-cycle codes to vary based on the connectivity and dropouts.

For the surface, colour and bivariate bicycle codes we consider, in the absence of dropouts, there are elegant constructions that allow us to measure all stabilisers in two layers, contracting and measuring half of the stabilisers in each. In this work, the process of finding these contraction assignments is automated, so that ACID rediscovers the deliberate constructions of the existing literature in the absence of dropouts and finds new schedules in their presence.

As depicted in Figure 2b, each stabiliser can usually be contracted and measured in multiple ways, and this redundancy can be exploited to measure stabilisers despite the presence of broken couplers. There are cases (e.g. the bottom panel of Figure 2b) where the presence of dropped couplers and/or qubits causes the support of stabilisers to be reduced, or even causes stabilisers to split into multiple parts. When this happens, the objects produced may no longer commute with each other, or with the undamaged stabilisers. In this case we deploy the theory of subsystem codes [17], allowing us to define product stabilisers made up of products of the anticommuting ‘quasi-stabilisers’ that do commute with one another.

3 Methods

ACID takes as input an undirected graph G whose nodes represent the qubits of our device, and whose edges represent the couplers between them, assuming perfect fabrication. I call this the ‘global connectivity graph’ or just ‘the connectivity’. It also takes a set of stabilisers defining the code, each of which is mapped to a subset of the nodes of the global connectivity graph. The subgraph of G induced by the nodes in the support of a given stabiliser is the ‘local connectivity graph’ of that stabiliser. Finally, we specify which qubits and couplers have dropped out.

The end-to-end process of going from a stabiliser code, global connectivity graph, and set of dropouts, to a syndrome extraction circuit and ultimately to the memory experiments described in this work is summarised in Figure 3.

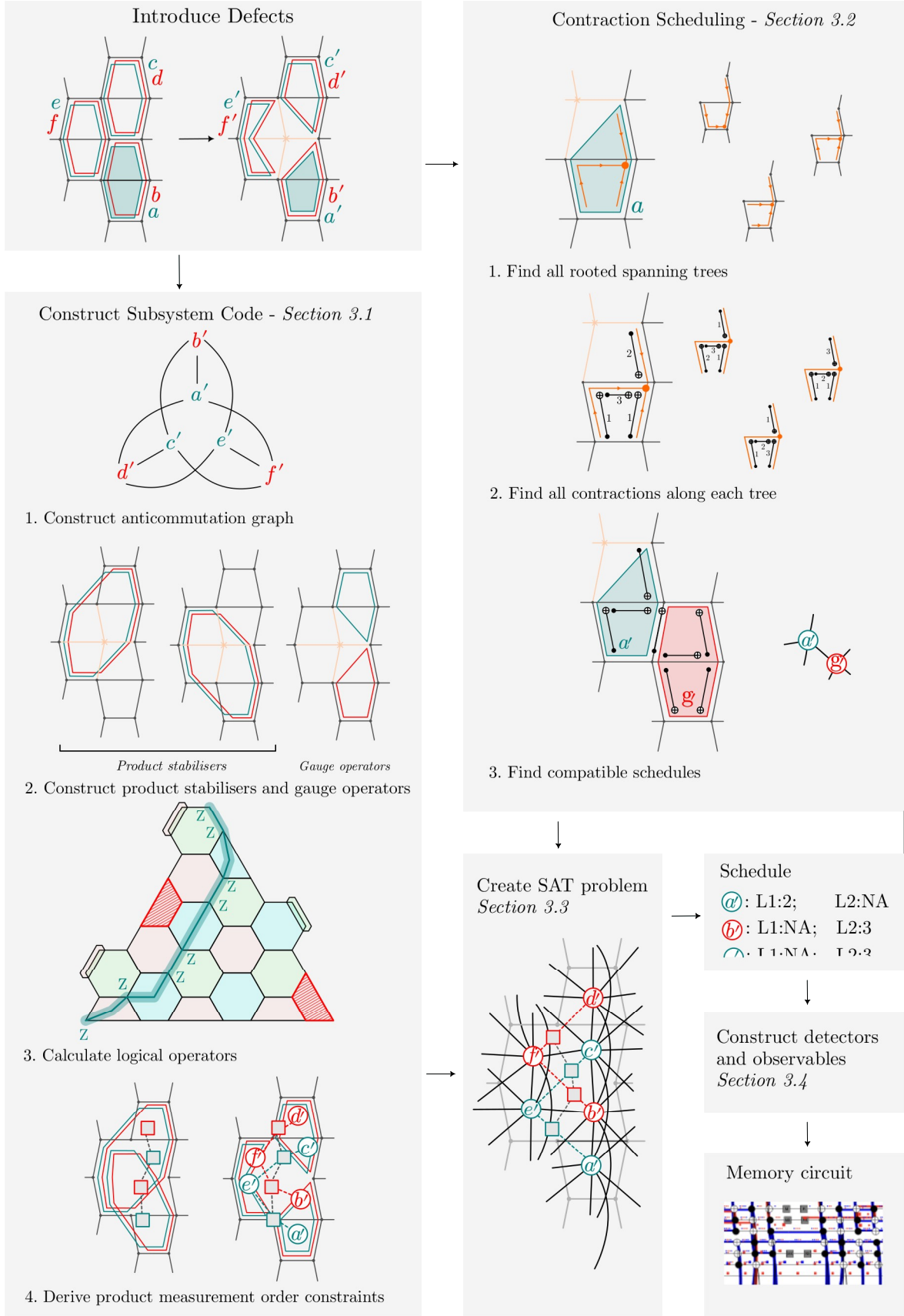


Figure 3: An overview of the process of compiling a memory circuit around defects.

3.1 Constructing a subsystem code from a defective stabiliser code

Each stabiliser is defined by its local connectivity graph. The first step of ACID is to split each stabiliser into one or more ‘quasi-stabiliser’ objects. For each stabiliser, it removes any edges corresponding to dropped couplers or nodes corresponding to dropped qubits from the local connectivity graph, leaving a modified graph with one or more connected components (or zero components if every qubit has dropped out). Each connected component defines a new operator, which I call a ‘quasi-stabiliser’.

In general, these quasi-stabilisers no longer all commute with one another. We could simply choose not to measure these anticommuting quasi-stabilisers at all, but we would in doing so be throwing away useful information we can use to catch errors, and we will end up substantially reducing the distance of our code, as well as changing the number of logical qubits encoded. We can in many cases maintain the code distance by packaging the anticommuting quasi-stabilisers into sets to form *product stabilisers*, which do all commute with each other. These product stabilisers are measured by physically contracting and measuring their constituent quasi-stabilisers (potentially over multiple syndrome extraction layers) and classically multiplying the results.

Define the quasi-stabiliser operators Q as CSS-type Pauli strings corresponding to the physical sets of connected qubits from which we build our code. I will in general notate elements of a set with lower-case letters, so that $Q = \{q_i\}$, and will sometimes interchangeably refer to operators and the physical constructions they describe, hopefully without ambiguity. The quasi-stabilisers, which include the still-commuting untouched stabilisers whose commutation relations were not altered by the dropouts, form a (non-commuting) *gauge group* that defines a subsystem code. In order to operate the subsystem code—that is, to construct from it syndrome extraction circuits—it is sufficient to find a basis for the subsystem code as follows.

Assuming the dropouts are sparsely distributed across the device, we will still have a set of untouched stabiliser generators $U \subseteq Q$ that still commute with everything in Q . The remaining ‘touched’ quasi-stabilisers $T = Q \setminus U$ need to be packaged into a maximal set of product stabilisers P , specified by a binary matrix B , and a set of exclusively anticommuting gauge operators G specified by a binary matrix C with

$$p_i = \prod t_j^{B_{ij}} \quad \text{and} \quad g_i = \prod t_j^{C_{ij}}. \quad (1)$$

Finally, we need a set of logical operators L that commute with all the operators defined so far, so that (subdividing $G = G^X \sqcup G^Z$ and $L = L^X \sqcup L^Z$) the following commutation rules hold for all i and j :

$$[u_i, u_j] = [u_i, p_j] = 0 \quad (2)$$

$$[g_i^X, g_j^Z] = [l_i^X, l_j^Z] = \delta_{ij} \quad (3)$$

$$[u_i, p_j] = [u_i, g_j] = [p_i, g_j] = [u_i, l_j] = [p_i, l_j] = [g_i, l_j] = 0 \quad (4)$$

where I have abused notation slightly in defining $[a, b] = 0$ where a and b commute and 1 where they anticommute. These admittedly dense relations are represented visually in Figure 4, where I also split our operator sets U and P into X and Z parts.

To construct all these operators, we first compute the binary anticommutation matrix A . A has a^X rows, equal to the number of X -type operators in T , and a^Z columns, equal to the number of Z -type operators in T . We set $A_{ij} = 1$ iff t_i anticommutes with t_j , so that we can view A as the biadjacency matrix of an ‘anticommutation graph’ as shown in Figure 3. We may then represent products of the X -type and Z -type quasi-stabilisers as binary vectors $v \in F_2^{a^X}$ and $w \in F_2^{a^Z}$, so that $vAw^T = 0$ if the corresponding product operators commute and $vAw^T = 1$ if they anticommute. We can bidiagonalise A using Gaussian elimination to obtain

$$VAW^T = \begin{bmatrix} I_g & 0 \\ 0 & 0 \end{bmatrix} \quad (5)$$

where g is the rank of A , and V and W are invertible matrices. Then the first g rows of V and of W are the exclusively anticommuting gauge operators we’re looking for, expressed as products of quasi-stabilisers. In other words, stacking the first g rows of V and W gives us the matrix C . The remaining rows of V and W are product stabilisers, forming the rows of the matrix B . We can then multiply out the definitions of the quasi-stabilisers to obtain the parity check matrices H^X and H^Z and gauge operator matrices G^X and G^Z . This set of product stabilisers is maximal and its span is unique. Although in principle error correction performance could be improved by choosing ‘nicer’ bases for the product operators, the algorithm is observed to generally produce product stabilisers that have low weight.

Once we have our stabilisers and gauge operators, we can construct the X (resp. Z) logical operators of the code by finding row vectors that commute with H^X and G^X (resp. H^Z and G^Z) operators whilst excluding the span of H^Z (resp. H^X). This can be done with a few applications of Gaussian elimination.

To measure a product stabiliser $p_i = t_a t_b \dots$, we may measure the constituent quasi-stabilisers simultaneously or sequentially. Either way, we require that at some point we collapse the qubits into a simultaneous eigenstate of the constituent operators. This is straightforward if the constituents are measured simultaneously (i.e. during the same contraction layer), but if they are measured sequentially, then we must ensure no quasi-stabiliser that anti-commutes with the constituents is measured in between the measurement of the first and last operators, lest the qubits be taken out of the shared eigenspace of the already-measured quasi-stabilisers. I show how to encode these requirements as an integer linear programming problem in Section 3.3.

3.2 Producing contraction schedules and finding scheduling constraints

The ancilla-free framework for error-correcting codes allows for a great deal of flexibility in how the mid-cycle stabilisers are measured. In this section, I describe a routine that takes as input a global connectivity graph and a set of quasi-stabilisers (Section 3.1), as well as a desired contraction step length t (e.g. $t = 2$ layers of CNOTs for the surface code) and produces an abstract scheduling graph. The nodes of the abstract scheduling graph represent quasi-stabilisers, each of which is decorated with a list of possible schedules for contracting that node, and each of whose edges is decorated with pairs of compatible schedules between the quasi-stabilisers represented by its two endpoints. The intra-layer constraints encoded in the scheduling graph, combined with the inter-layer requirements resulting from the structure of the subsystem code (Section 3.1), give us the full SAT problem (Section 3.3) we solve and optimise over to give us full multi-layer schedules that define a syndrome extraction round.

The scheduling problem is highly symmetric: all stabiliser codes considered in this work consist of a limited number of stabiliser ‘shapes’ that tile to produce a code. In my implementation, I leverage this fact by caching these shapes and calculating both the contraction schedules and the constraints between them only for unique cases. Here, I simplify the presentation, and provide these details via the source code.

First, we wish to calculate every possible contraction schedule for each quasi-stabiliser. Each quasi-stabiliser has an associated ‘local connectivity graph’, which is the subgraph of the global connectivity graph induced by its nodes. A contraction schedule is a series of CNOTs, each on some edge of the local graph, over t timesteps, that contracts a Pauli so that it is supported on only one root qubit.

Contraction schedules can be uniquely specified by 1) a rooted-spanning-tree subgraph of the local graph, and 2) a ‘timing’: for each edge e , a timestep $1 \leq t_e \leq t$, such that no two edges that share a node are contracted at the same time, and contractions over edges more distant from the root occur before those closer to the root. In the case of X -type quasi-stabilisers, this means that the first CNOTs to occur are targeted on leaf nodes of the tree, and remove the support of the Pauli string on the leaf-node qubits. Subsequent CNOTs then recede the support further up the tree until it is localised at the root. Note that some rooted spanning trees have no valid timings. The spanning trees can be enumerated in linear time using a standard algorithm [19]. The timings can also be enumerated in linear time using a simple recursive algorithm (see the source).

We then consider every pair of quasi-stabilisers q_a, q_b that share at least one qubit. For each pair, we consider every pair of contraction schedules c_a, c_b , and decide if they are compatible according to the following criteria:

1. the CNOTs in c_a and c_b act on disjoint qubits in each timestep, excepting the case where the two CNOTs are the same operation.
2. Neither schedule is permitted to modify the support of Pauli string of the other quasi-stabiliser. For example, an X -type quasi-stabiliser q_a sharing a single qubit with another X -type quasi-stabiliser q_b could not have a schedule a containing in the first timestep a CNOT controlled on the common qubit, as this would copy the support of the q_b ’s Pauli string into itself, preventing b from being able to contract the string onto the chosen root qubit.

To enforce the second criterion, we precalculate the evolution of the Pauli string through each schedule and compare it with the CNOTs of other schedules as necessary. Any pair c_a, c_b satisfying the above will contract the Pauli strings onto their respective roots simultaneously. Note that any two anticommuting quasi-stabilisers can never have any compatible contraction schedules, as the contraction process is unitary and hence preserves (anti)commutation of observables; but the contraction process is required to ‘separate’ the Pauli strings onto

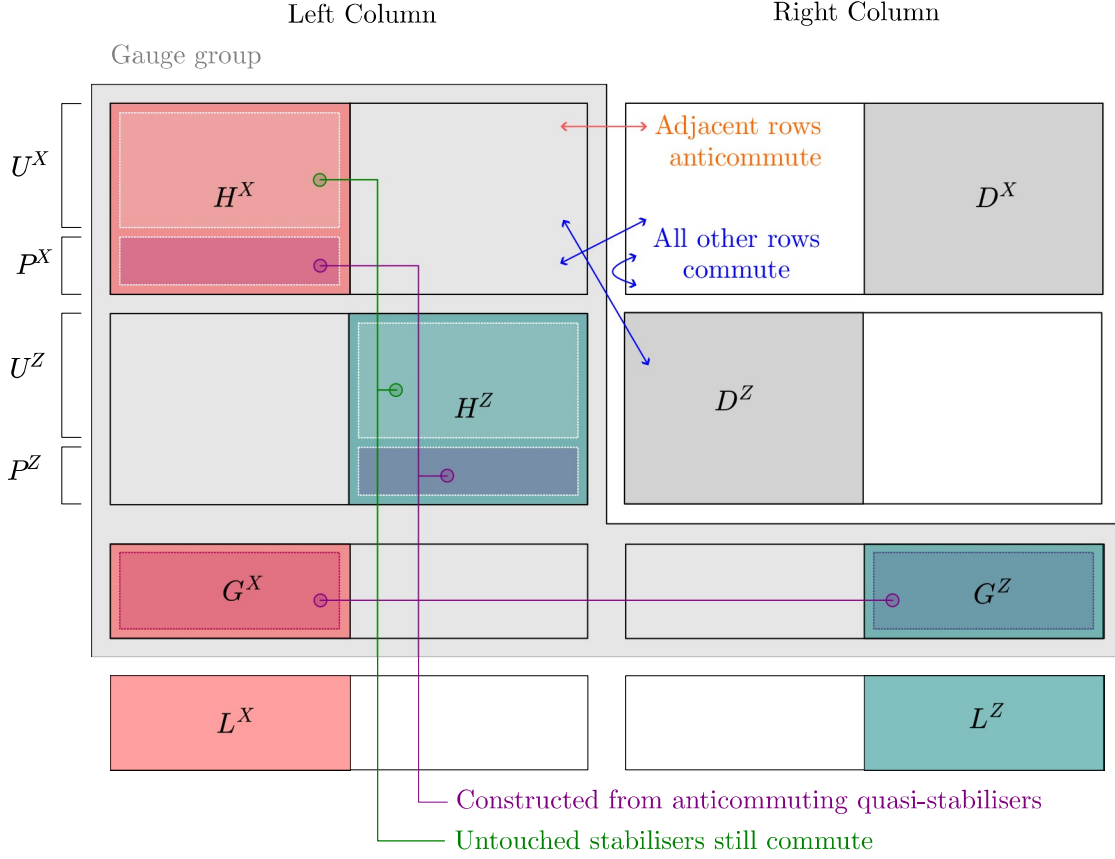


Figure 4: *The commutation relations of a CSS subsystem code (symplectic basis).* The left and right ‘columns’ are each binary matrices, each of width $2n$ (twice the number of physical qubits) and height n , so that, taken together, the $2n$ rows form a complete basis for F_2^{2n} . Each row of either column represents a Pauli operator. Within each row the first n bits represent the places where a Pauli operator contains an X , and the last n bits represent where it contains a Z . Then the diagram can be read as a series of left-right pairs of Pauli operators, so that each operator anticommutes with its neighbour and commutes with all others. Within this representation, there are blocks corresponding to the parity check matrices H^X and H^Z , the gauge operators G^X and G^Z , and logical operators L^X and L^Z . To complete the basis, I’ve included the destabiliser matrices D^X and D^Z , although for practical purposes we do not need to compute these. The bulk of the stabilisers are the existing untouched stabilisers U^X and U^Z , with the remaining stabilisers and gauge operators formed as products of the ‘touched’ quasi-stabilisers. Viewed this way, it is clear that a subsystem code can be seen as a stabiliser code with an arbitrary partition of the logical qubits into both logical qubits and gauge qubits. The other definition of a subsystem code, via a gauge group, is illustrated with a grey box. (Note - this representation ignores phases, which we would need to include if we were to write a full stabiliser tableau—see [18])

disjoint root qubits, which always commute. The scheduling graph will in these cases have an edge connecting the two quasi-stabilisers decorated with an empty list containing no allowed schedules.

By checking all pairs against these criteria, ACID produces a ‘scheduling graph’ that represents all the constraints that prevent two quasi-stabilisers from being contracted simultaneously with given schedules.

3.3 Constructing an integer linear programming problem

Having constructed a subsystem code from our defective stabiliser code and calculated all the physical constraints on contraction schedules for neighbouring quasi-stabilisers, ACID builds an integer linear programming (ILP) problem that can be fed into a solver to find schedules.

Ideally, the final schedule will use as few layers per round as possible, so we start by constructing a problem with $L = 2$ layers, and increase L if the problem is found to be infeasible by the solver.

ACID defines variables corresponding to each schedule on each quasi-stabiliser on each layer, and a further set of variables that encode the product stabilisers of our subsystem code (Section 3.1). It then encodes into our problem (see Appendix B for details) the constraints:

- no two schedules are chosen on neighbouring quasi-stabilisers within the same layer that are incompatible (physical constraints);
- no quasi-stabiliser is measured that would randomise the result of the measurement of a product stabiliser currently being measured.

ACID solves the problem using CP-SAT [20], which is known to handle large-scale scheduling problems well. The solver is given the task of trying to minimise the number of layers required to measure all the stabilisers at least once over a round. As a secondary heuristic optimisation, it tries to maximise the number of times each stabiliser is measured. The output of the solver gives us L , and a ‘global schedule’—a recipe for which quasi-stabilisers are to be measured in which layers and using which contraction schedule.

3.4 Assigning detectors

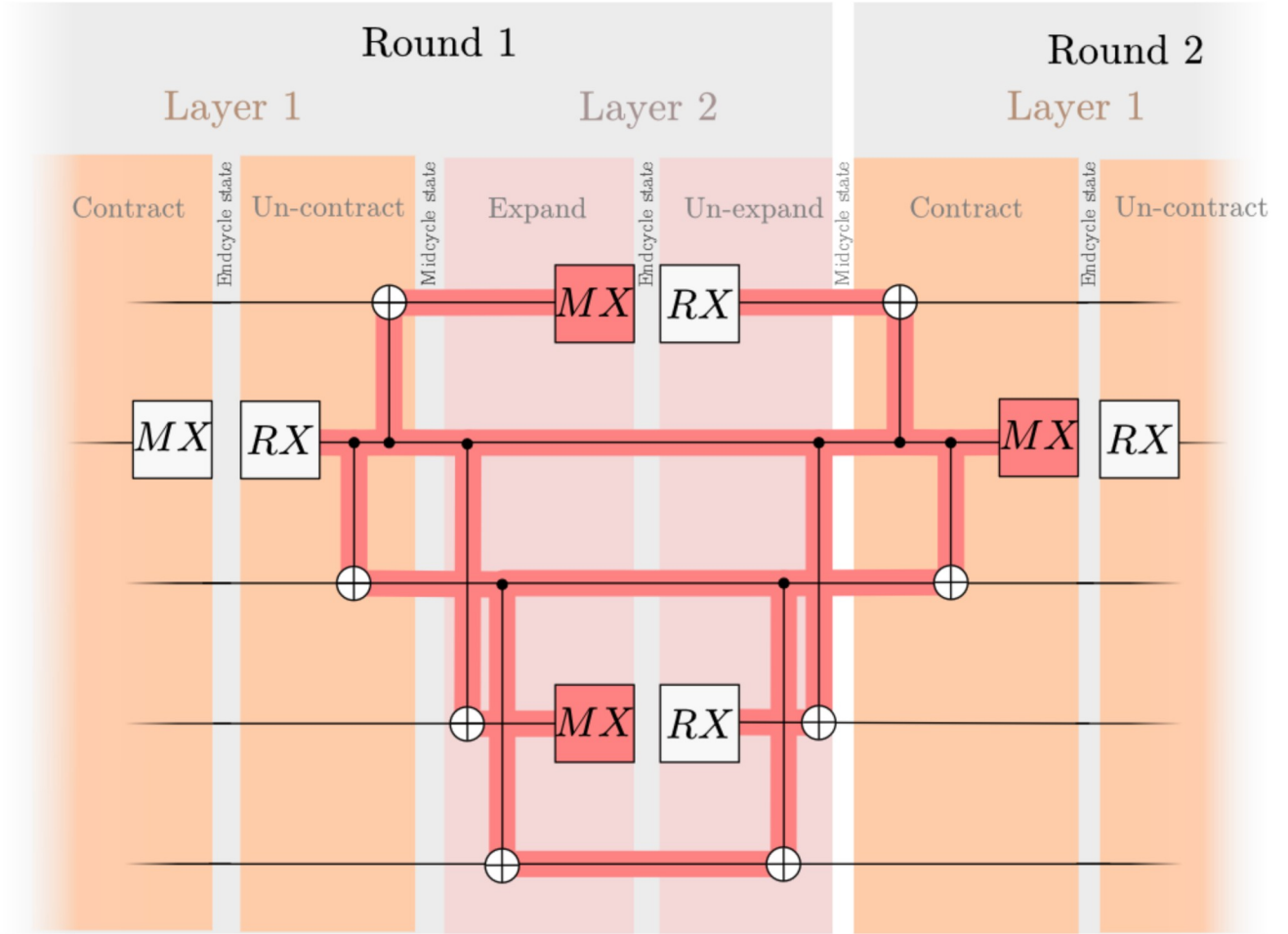


Figure 5: *Detector assignment in the ancilla-free paradigm.* Depicted is a portion of two rounds of syndrome extraction, each of which has $L = 2$ layers. A detector is defined between two rounds, being contracted in the first layer of both. The detector includes the final measurement, but also measurements of the stabilisers expanded support in the intervening second layer.

The primary output of ACID is a syndrome extraction circuit, which in practice would be performed at regular intervals throughout a quantum computation. To test its performance in the present work I simulate memory experiments consisting of R rounds of syndrome extraction, where R is set to the expected code-capacity distance of the end-cycle codes in the dropout-free case (see Section 5 for the different codes). Each of the R rounds consists of L layers wherein some subset of the quasi-stabilisers are contracted and measured.

Stim [21], the Clifford simulator used here for the memory experiments, defines ‘detectors’ as sets of measurements whose overall parity is deterministic when no errors occur: violations of these expected parities are reported to the decoder. It is the task of the decoder to infer from the detector results which errors occurred in the circuit.

It is most helpful to think of these detectors in terms of Pauli flows [22]. For our purposes, Pauli flows are Pauli strings that propagate through our circuit in the usual way, where we can add to the support of the Pauli string (‘seed the flow’) and where we can reduce the support of the Pauli strings (‘terminate the flow’) on measurements in the correct basis. Any Pauli flow that is both seeded and terminated within the circuit can be used to define a valid detector.

Consider the case of a simple non-product stabiliser consisting of a single quasi-stabiliser. Suppose the stabiliser is contracted in layer A and then again in layer B . This case is illustrated in Figure 5. Layers A and B may lie in different rounds of syndrome extraction or they may both lie in the same round if the scheduler has

managed to include extra redundant contractions of the stabiliser within one round. (In the case of Figure 5, ‘layer A ’ refers to layer 1 of round 1, and ‘layer B ’ refers to layer 1 of round 2.) This means that in both layers A and B , the stabiliser is contracted to some root qubit, which is measured and reset, and then un-contracted. In general, the root qubits onto which the stabiliser is contracted may not be the same in A and B . We imagine a Pauli flow seeded on the root qubit reset instruction in layer A . The string, which starts as a single X or Z , un-contracts in the second half of layer A to have exactly the support of the stabiliser in the mid-cycle code. In any layers between A and B , the string expands, where in general some of the qubits in the support of the expanded operator are measured and reset. Note that these measurements will always be in the correct basis—if one wasn’t, it would imply that our stabilisers don’t all commute. Since the Pauli flow cannot terminate on these resets that occur during the expansion layers, we must include the measurements that precede them and re-seed the Pauli flow on the resets. This allows the Pauli string to ‘flow through’ the measurement and reset and then un-expand and contract onto a single qubit in B , where we terminate the Pauli flow by measuring that qubit. ACID adds detectors of this form between each contraction and the next.

Defining detectors for product stabilisers, each made up of multiple anti-commuting quasi-stabilisers, is more complicated. A single measurement of the value of a product stabiliser might take multiple layers. Define a ‘completion’ for a product stabiliser as a series of layers over which all of the constituent quasi-stabilisers of a product stabiliser are contracted and measured. We are guaranteed by the solver to have at least one completion per round of syndrome extraction. Detectors are then defined between each completion event and the next. To define a detector between completion A and completion B , we imagine a Pauli flow seeded on each root-qubit reset in the contractions that comprise completion A . We track the Pauli string resulting from these seeds, noting that in general they interfere with one another. Then as in the non-product case, we must terminate and re-seed the Pauli flow wherever the expanded support of the product stabiliser is measured. Finally, we terminate the flow with the measurements of root qubits that correspond to the measurements comprising completion B .

We can also add some extra detectors where a quasi-stabiliser is measured more than twice over two completions, and there is no anticommuting quasi-stabiliser that randomises its value in between two of them. Also note that to simulate a memory experiment, we add extra detectors between each contraction and the initial and final Pauli product measurements we use to put the qubits into the codespace and measure out at the end.

Stim observables, the circuit-level analogue of logical operators, are also defined as sets of measurements whose overall parity is deterministic in the absence of errors, and can be specified using Pauli flows. In the language of ancilla-free QEC, we can think of a logical operator as an operator that is never contracted. It is therefore expanded in every layer of every round, and we must track its expansion and terminate and re-seed the flow accordingly. In general, expanding a stabiliser or observable can reduce its support, so contraction circuits can be distance-reducing. For more details of the memory experiments I constructed for our simulations see Appendix C.

4 The surface code

The state of the art for handling dropouts in the surface code is the LUCI framework [9], which was the original inspiration for this work. LUCI offers an intuitive way of measuring all quasi-stabilisers over $L = 4$ contraction layers within a round of syndrome extraction. The technique presented in the original paper relied on the flexibility in contraction schedules offered by a (degree-4) square-grid connectivity to deal with dropped couplers and resorted to forming product stabilisers to deal with dropped qubits, with certain pathological patterns of dropped couplers treated as dropped qubits. The more recent work [10] allows LUCI to be applied to a degree-3 hexagonal/brickwork-style layout by assigning product stabilisers as soon as couplers are dropped (as well as some scheduling improvements). In both works, couplers can often be dropped without reducing the code-capacity distance d of the code. In particular, in the square-grid case the loss of an isolated coupler can always be accommodated without losing distance, and in the hex-grid case we can tolerate the loss of about a third of the couplers without losing distance, with the distance reducing by one for the other couplers.

Applying ACID to both the square-grid and hex-grid layouts, I find that ACID can often compile a syndrome extraction circuit with $L = 2$ layers and in all cases I tried (up to the loss of 3 qubits and 3 couplers), can create a circuit with $L = 3$. In principle this should allow the time between lattice surgery operations to be reduced from $4d$ using LUCI to $3d$ or even $2d$ with sufficient quality fabrication and device postselection. In Figure 7, we show the effect on L , d and the logical error rate of dropping out any single coupler or qubit in a distance-11 patch of square-grid and hex-grid surface code. As with LUCI, the code-capacity distance (Appendix A) is often preserved, with the square-grid connectivity unsurprisingly more tolerant to dropped couplers than

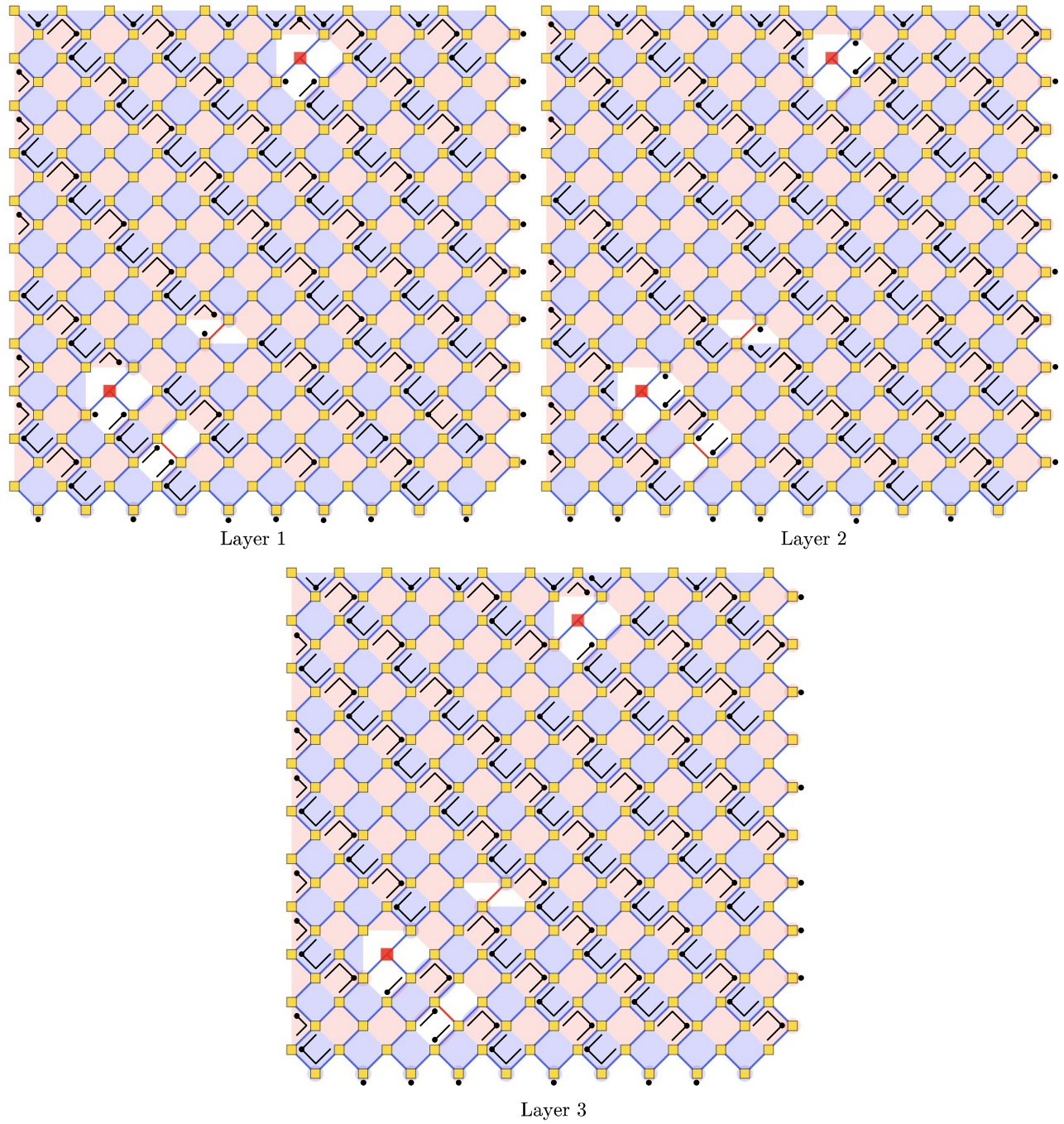


Figure 6: A LUCI-like diagram showing a syndrome extraction circuit of the brickwork surface code in the presence of two dropped qubits and two dropped couplers consisting of three contraction layers. I've followed the notation of [9], where stabilisers that are contacted in a given layer are indicated with a black depiction of the rooted spanning tree used to contract them. Dropped qubits and couplers are indicated in red. [Click here to open the circuit in Shatter.](#)

the hex-grid connectivity. The relative performance of LUCI and ACID is summarised in Figure 1, and I give a full characterisation of the performance of ACID and LUCI under different numbers of dropped qubits and couplers in Appendix D.

An important caveat is that ACID sometimes produces detector error models that do not admit ready approximation as decoding graphs, meaning we lose access to fast matching decoders. I believe that this is a result of overlapping redundant measurements of quasi-stabilisers, and can be fixed in future work by imposing additional constraints on the scheduling graph.

5 Bivariate bicycle codes

Notwithstanding discussions of possible theoretical methods for doing so [23, 24], I believe ACID is the first concrete proposal for handling dropouts in the bivariate bicycle (BB) codes [25]. Following [11], I applied ACID to the 144-qubit and 288-qubit codes, each with 12 logical qubits, from [25] as mid-cycle codes. In the dropout-free case, these mid-cycle codes produce end-cycle codes with distance 6 and distance 12 respectively (these are the distances I use as the length of the memory experiments).

I applied ACID to these codes with two underlying connectivities, inspired by the hex-grid and square-grid surface code connectivities: first, the degree-5 connectivity introduced in [11], and second a ‘hexagonal’ connectivity in which the six qubits defining a stabiliser are connected in a cycle (see Figure 8). This latter connectivity is degree-6, but offers more flexibility in contraction schedules for each stabiliser. In particular, where in the degree-5 connectivity the loss of any coupler results in anticommuting quasi-stabilisers (as the local connectivity graph is a tree), the cyclic nature of the hexagonal connectivity means we can tolerate the loss of isolated couplers whilst still being able to measure all the original stabilisers. It is interesting to note that for some BB codes, such as the 288 code, the hexagonal connectivity is completely symmetrical in its couplers and qubits (the associated undirected graph is arc-transitive), whereas in others such as the 144 code the edges split into two or more edge orbits.

The non-locality and complexity of these connectivities mean that neither will likely prove a viable layout for practical quantum hardware. Future work should examine the dropout tolerance of proposals such as [26, 27] that aim for more hardware-friendly QLDPC codes.

The full data for the 144-qubit and 288-qubit BB codes for up to 3 dropped qubits and couplers is shown in Appendix D. For the hexagonal connectivity, in all the scenarios tested, the solver finds a schedule with $L = 4$ or fewer. Using the more limited degree-5 connectivity, the solver occasionally needs $L = 5$ for the 144 code.

Of particular interest is the single-dropped-coupler case for the 144-qubit code with the hexagonal connectivity (Appendix D). I note that the introduction of a single defective coupler results in most cases in a substantially lower logical error rate than in the dropout-free case. Investigating further, one can calculate the distance of the end-cycle codes resulting from a given schedule and see that certain contraction schedules lead to an end-cycle-distance of 7 or even 8. Alternative contraction schedules with higher distances are also discussed (in the degree-5, dropout-free case) in the appendix of [11]. It is noteworthy that the dropout-free schedules are tightly bunched in logical error rate, suggesting that the heuristic metrics that the solver optimises for in fact push it into a schedule that is sub-optimal, and that the frustration introduced by a new constraint allows the solver to explore better options.

Whereas for the surface code lattice surgery is straightforwardly generalised to the case of patches with dropouts, applying ACID to the BB codes corrodes the symmetry of the code that is used in proposals for logical processing with the code. In particular, the code is no longer translationally invariant, so we appear to lose access to a large class of automorphisms used in [28] to perform universal computation.

6 The colour code

I applied ACID to three flavours of colour code, with all data shown in Appendix D. The first, the degree-3 ancilla-free colour code is taken directly from [12] (where it is called the ‘middle-out’ circuit). This connectivity is naturally implemented on a hex-grid qubit layout. The local connectivity graphs in this connectivity are then hexagons, so we expect to often be able to measure the original stabilisers in the presence of dropped couplers. However, the tightly packed and overlapping nature of the colour code stabilisers means that the global scheduling problem is very sensitive to local restrictions, and a single dropped coupler is normally enough to force the scheduler to require four contraction layers.

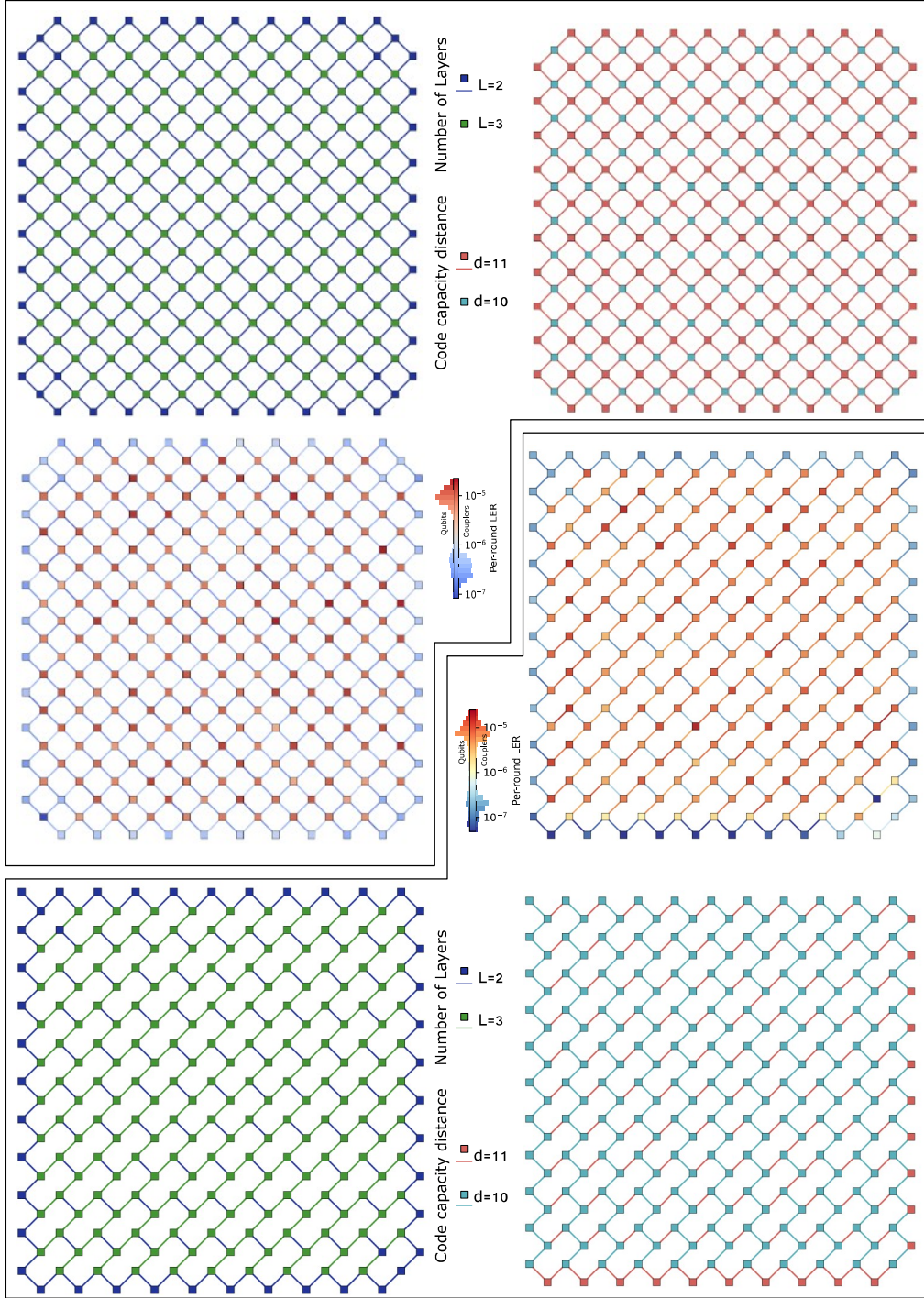


Figure 7: *The effect of dropping a single qubit or coupler from the surface code in square-grid and hex-grid configuration.* The top-left half of the figure shows the effect on the number of layers required for a syndrome extraction circuit L (top-left), the mid-cycle code capacity distance (top-right), and logical error rate (middle-left). The bottom-right half of the figure shows the same for the hex-grid surface code. In each case, each qubit and coupler is coloured according to the effect that the loss of that qubit or coupler has on the relevant quantity. For the logical error rate (LER), the central colour-bars are equipped with histograms on the left and right corresponding to the the distribution of the dropped-qubit (left) and dropped-right (right) LER distribution.

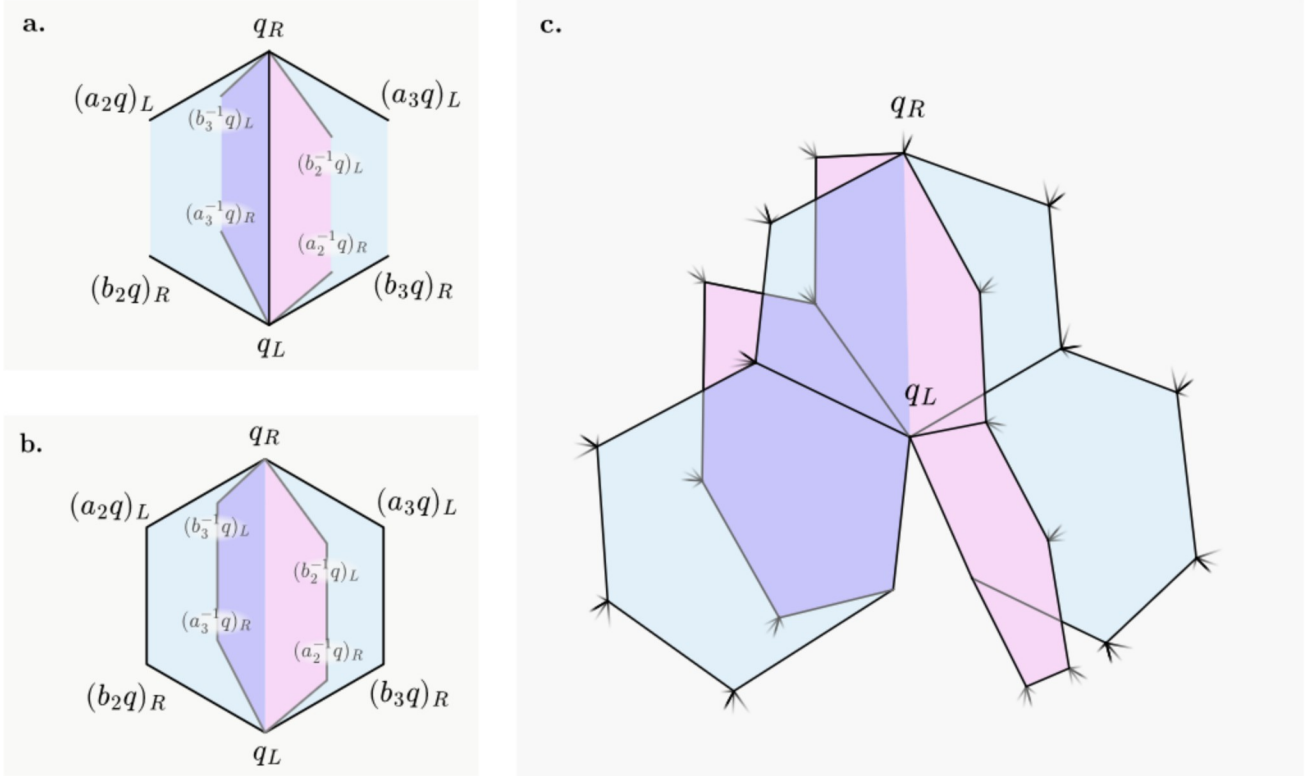


Figure 8: *Connectivities for the bivariate bicycle (BB) code.* Vertices are qubits, and solid lines between them represent the presence of couplers. I’ve used the standard polynomial specification of BB codes where qubits are labelled as $q_{L/R}$, with q a monomial in a finite polynomial ring and a left (‘L’) or ‘right’ (‘R’) label. **a.** The local connectivities of X (blue) and Z (pink/purple) stabilisers in construction taken from [11]. This construction results in 5-regular global connectivity graph. **b.** The local connectivity of the same stabilisers in my hexagonal connectivity. Note that the qubits associated with monomials q_L and q_R are no longer connected, but we introduce two new edges with directions $a_2 b_2^{-1}$ and $a_3 b_3^{-1}$. **c.** All of the stabilisers containing a given L qubit in the hexagonal connectivity, showing the 6-regularity of the graph.

The second connectivity I tried is what results if you take the same middle-out colour code circuit but lay out the qubits on a full square lattice, meaning we introduce an extra redundant edge crossing from the two extremes of each local hexagon. This ‘square-grid ancilla-free’ colour code is the example used in Figure 3. The extra redundancy introduced by this edge allows the solver to tolerate a single dropped coupler in three or fewer layers in the majority of cases.

Finally, I tried a version of the colour code based on the ‘superdense’ syndrome extraction circuit also from [12]. Eight qubits are assigned to each stabiliser of the colour code, six outer qubits and two inner ones (these inner qubits were previously measurement ancillae). I then give ACID the task of scheduling four stabilisers per stabiliser of the colour code: an X and a Z weight-8 stabiliser and X and Z weight-2 stabilisers supported on the inner qubits. The natural solution found by the solver is then almost exactly the superdense syndrome extraction circuit from the literature, but whereas the original superdense circuit has three ‘incoming’ CNOTs preceding three ‘outgoing’ CNOTs, in this circuit the two groups take turns to go first. This leads to the same number of detectors, but they are more localised in time, suggesting the decoding performance may be better. This superdense colour code is embedded on a full square-grid lattice. This means we have two redundant edges in the local connectivity graphs of the weight-8 stabilisers. This redundancy, combined with the fact that the contraction circuits take four timesteps, can lead to a combinatorial explosion in the number of contraction schedules, overwhelming the solver (even though in principle having lots of schedules to choose from is a good thing). To overcome this, we prune the contraction schedules ahead of feeding them to the solver based on their compatibility with certain ‘preferred schedules’.

7 Outlook

ACID can be applied to any CSS stabiliser code with any connectivity. Even once components have dropped out, there are many possible syndrome extraction circuits to choose from. ACID uses crude heuristics as tie-breaks, preferring global schedules that ‘cram in’ as many stabiliser contractions as possible. Some global schedules will needlessly reduce the circuit distance of the quantum memory circuit when others compatible with the same dropped qubits and couplers don’t. This is best illustrated in the case of the 144-qubit bivariate bicycle code, where the introduction of a single dropped coupler forces the solver to output global schedules with a lower logical error rate than those it finds in the dropout-free case. Future work should address this, introducing additional constraints, analytic or heuristic, into the solver to force it to select better schedules. Other requirements could be introduced in this way: in principle, one could vary the syndrome extraction circuit between rounds, measuring and resetting all qubits with some frequency in order to mitigate leakage.

Whilst the basic ‘merge and split’ operations of surface code and colour code lattice surgery [29, 30] should still be valid in the presence of dropouts, it is not clear how the loss of symmetry of codes in the presence of dropouts will affect logical operations for more general codes.

This work also has implications for the co-design of quantum hardware and error-correcting codes. Different hardware connectivities should be considered not just for the codes they support, but the flexibility they provide in case of imperfect fabrication. There may be design elements that can be introduced that substantially improve this flexibility without a correspondingly large overhead in qubit count or connectivity.

8 Acknowledgements

I’m hugely grateful to my supervisors, Dan Browne and Earl Campbell, for their support and sage advice throughout the completion of this work, and their comments on various drafts of this manuscript.

This work was supported by Riverlane and the Engineering and Physical Sciences Research Council [grant number EP/Y035046/1].

References

- [1] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C. Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando G. S. L. Brandao, David A. Buell, et al. “Quantum Supremacy Using a Programmable Superconducting Processor”. In: *Nature* 574.7779 (Oct. 2019), pp. 505–510. ISSN: 1476-4687. DOI: 10.1038/s41586-019-1666-5. URL: <https://www.nature.com/articles/s41586-019-1666-5> (visited on 11/25/2025).
- [2] Rajeev Acharya, Dmitry A. Abanin, Laleh Aghababaie-Beni, Igor Aleiner, Trond I. Andersen, Markus Ansmann, Frank Arute, Kunal Arya, Abraham Asfaw, Nikita Astrakhantsev, et al. “Quantum Error Correction below the Surface Code Threshold”. In: *Nature* 638.8052 (Feb. 2025), pp. 920–926. ISSN: 1476-4687. DOI: 10.1038/s41586-024-08449-y. URL: <https://www.nature.com/articles/s41586-024-08449-y> (visited on 11/25/2025).
- [3] James M. Auger, Hussain Anwar, Mercedes Gimeno-Segovia, Thomas M. Stace, and Dan E. Browne. “Fault-Tolerance Thresholds for the Surface Code with Fabrication Errors”. In: *Physical Review A* 96.4 (Oct. 12, 2017), p. 042316. ISSN: 2469-9926, 2469-9934. DOI: 10.1103/PhysRevA.96.042316. arXiv: 1706.04912 [quant-ph]. URL: <http://arxiv.org/abs/1706.04912> (visited on 11/16/2025).
- [4] Adam Siegel, Armands Strikis, Thomas Flatters, and Simon Benjamin. “Adaptive Surface Code for Quantum Error Correction in the Presence of Temporary or Permanent Defects”. In: *Quantum* 7 (July 25, 2023), p. 1065. DOI: 10.22331/q-2023-07-25-1065. URL: <https://quantum-journal.org/papers/q-2023-07-25-1065/> (visited on 11/16/2025).
- [5] Armands Strikis, Simon C. Benjamin, and Benjamin J. Brown. “Quantum Computing Is Scalable on a Planar Array of Qubits with Fabrication Defects”. In: *Physical Review Applied* 19.6 (June 29, 2023), p. 064081. DOI: 10.1103/PhysRevApplied.19.064081. URL: <https://link.aps.org/doi/10.1103/PhysRevApplied.19.064081> (visited on 11/16/2025).

- [6] Sophia Fuhui Lin, Joshua Vizslai, Kaitlin N. Smith, Gokul Subramanian Ravi, Charles Yuan, Frederic T. Chong, and Benjamin J. Brown. “Codesign of Quantum Error-Correcting Codes and Modular Chiplets in the Presence of Defects”. In: *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. Apr. 27, 2024, pp. 216–231. DOI: 10.1145/3620665.3640362. arXiv: 2305.00138 [quant-ph]. URL: <http://arxiv.org/abs/2305.00138> (visited on 11/16/2025).
- [7] Catherine Leroux, Sophia F. Lin, Przemyslaw Bienias, Krishanu R. Sankar, Asmae Benhemou, Aleksander Kubica, and Joseph K. Iverson. “Snakes and Ladders: Adapting the Surface Code to Defects”. In: *PRX Quantum* 6.4 (Oct. 2, 2025), p. 040302. ISSN: 2691-3399. DOI: 10.1103/815q-xjrb. arXiv: 2412.11504 [quant-ph]. URL: <http://arxiv.org/abs/2412.11504> (visited on 01/10/2026).
- [8] Campbell McLauchlan, György P. Gehér, and Alexandra E. Moylett. “Accommodating Fabrication Defects on Floquet Codes with Minimal Hardware Requirements”. In: *Quantum* 8 (Dec. 12, 2024), p. 1562. DOI: 10.22331/q-2024-12-12-1562. URL: <https://quantum-journal.org/papers/q-2024-12-12-1562/> (visited on 01/10/2026).
- [9] Dripto M. Debroy, Matt McEwen, Craig Gidney, Noah Shutt, and Adam Zalcman. *LUCI in the Surface Code with Dropouts*. Version 1. Oct. 18, 2024. DOI: 10.48550/arXiv.2410.14891. arXiv: 2410.14891 [quant-ph]. URL: <http://arxiv.org/abs/2410.14891> (visited on 11/16/2025). Pre-published.
- [10] Oscar Higgott, Benjamin Anker, Matt McEwen, and Dripto M. Debroy. *Handling Fabrication Defects in Hex-Grid Surface Codes*. Aug. 11, 2025. DOI: 10.48550/arXiv.2508.08116. arXiv: 2508.08116 [quant-ph]. URL: <http://arxiv.org/abs/2508.08116> (visited on 11/16/2025). Pre-published.
- [11] Mackenzie H. Shaw and Barbara M. Terhal. “Lowering Connectivity Requirements For Bivariate Bicycle Codes Using Morphing Circuits”. In: *Physical Review Letters* 134.9 (Mar. 4, 2025), p. 090602. ISSN: 0031-9007, 1079-7114. DOI: 10.1103/PhysRevLett.134.090602. arXiv: 2407.16336 [quant-ph]. URL: <http://arxiv.org/abs/2407.16336> (visited on 11/16/2025).
- [12] Craig Gidney and Cody Jones. *New Circuits and an Open Source Decoder for the Color Code*. Dec. 14, 2023. DOI: 10.48550/arXiv.2312.08813. arXiv: 2312.08813 [quant-ph]. URL: <http://arxiv.org/abs/2312.08813> (visited on 11/16/2025). Pre-published.
- [13] *Riverlane/ACID*. GitHub. URL: <https://github.com/riverlane/ACID> (visited on 11/28/2025).
- [14] Benjamin Anker and Dripto M. Debroy. *Optimized Measurement Schedules for the Surface Code with Dropout*. Dec. 11, 2025. DOI: 10.48550/arXiv.2512.10871. arXiv: 2512.10871 [quant-ph]. URL: <http://arxiv.org/abs/2512.10871> (visited on 01/11/2026). Pre-published.
- [15] Michael Vasmer and Aleksander Kubica. “Morphing Quantum Codes”. In: *PRX Quantum* 3.3 (Aug. 8, 2022), p. 030319. ISSN: 2691-3399. DOI: 10.1103/PRXQuantum.3.030319. arXiv: 2112.01446 [quant-ph]. URL: <http://arxiv.org/abs/2112.01446> (visited on 11/25/2025).
- [16] Matt McEwen, Dave Bacon, and Craig Gidney. *Relaxing Hardware Requirements for Surface Code Circuits Using Time-dynamics*. arXiv.org. Feb. 4, 2023. DOI: 10.22331/q-2023-11-07-1172. URL: <https://arxiv.org/abs/2302.02192v2> (visited on 11/16/2025).
- [17] Dave Bacon. “Operator Quantum Error Correcting Subsystems for Self-Correcting Quantum Memories”. In: *Physical Review A* 73.1 (Jan. 30, 2006), p. 012340. ISSN: 1050-2947, 1094-1622. DOI: 10.1103/PhysRevA.73.012340. arXiv: quant-ph/0506023. URL: <http://arxiv.org/abs/quant-ph/0506023> (visited on 11/25/2025).
- [18] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th anniversary edition. Cambridge: Cambridge university press, 2010. ISBN: 978-1-107-00217-3.
- [19] Pawel Winter. “An Algorithm for the Enumeration of Spanning Trees”. In: *BIT Numerical Mathematics* 26.1 (Mar. 1, 1986), pp. 44–62. ISSN: 1572-9125. DOI: 10.1007/BF01939361. URL: <https://doi.org/10.1007/BF01939361> (visited on 11/04/2025).
- [20] *Google/or-Tools*. Google, Nov. 4, 2025. URL: <https://github.com/google/or-tools> (visited on 11/04/2025).
- [21] Craig Gidney. “Stim: A Fast Stabilizer Circuit Simulator”. In: *Quantum* 5 (July 6, 2021), p. 497. ISSN: 2521-327X. DOI: 10.22331/q-2021-07-06-497. arXiv: 2103.02202 [quant-ph]. URL: <http://arxiv.org/abs/2103.02202> (visited on 11/16/2025).

- [22] Daniel E Browne, Elham Kashefi, Mehdi Mhalla, and Simon Perdrix. “Generalized Flow and Determinism in Measurement-Based Quantum Computation”. In: *New Journal of Physics* 9.8 (Aug. 2007), p. 250. ISSN: 1367-2630. DOI: 10.1088/1367-2630/9/8/250. URL: <https://doi.org/10.1088/1367-2630/9/8/250> (visited on 11/16/2025).
- [23] Yichen Xu and Arpit Dua. *Fault-Tolerant Protocols through Spacetime Concatenation*. July 2, 2025. DOI: 10.48550/arXiv.2504.08918. arXiv: 2504.08918 [quant-ph]. URL: <http://arxiv.org/abs/2504.08918> (visited on 11/14/2025). Pre-published.
- [24] Runshi Zhou, Fang Zhang, Hui-Hai Zhao, Feng Wu, Linghang Kong, and Jianxin Chen. *Lowvre: Relaxing Hardware Requirements of Quantum LDPC Codes by Routing with Expanded Quantum Instruction Set*. Version 1. Aug. 29, 2025. DOI: 10.48550/arXiv.2508.20858. arXiv: 2508.20858 [quant-ph]. URL: <http://arxiv.org/abs/2508.20858> (visited on 11/14/2025). Pre-published.
- [25] Sergey Bravyi, Andrew W. Cross, Jay M. Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J. Yoder. “High-Threshold and Low-Overhead Fault-Tolerant Quantum Memory”. In: *Nature* 627.8005 (Mar. 2024), pp. 778–782. ISSN: 1476-4687. DOI: 10.1038/s41586-024-07107-7. URL: <https://www.nature.com/articles/s41586-024-07107-7> (visited on 11/16/2025).
- [26] Jens Niklas Eberhardt, Francisco Revson F. Pereira, and Vincent Steffan. *Pruning qLDPC Codes: Towards Bivariate Bicycle Codes with Open Boundary Conditions*. Dec. 5, 2024. DOI: 10.48550/arXiv.2412.04181. arXiv: 2412.04181 [quant-ph]. URL: <http://arxiv.org/abs/2412.04181> (visited on 11/16/2025). Pre-published.
- [27] Zijian Liang, Jens Niklas Eberhardt, and Yu-An Chen. “Planar Quantum Low-Density Parity-Check Codes with Open Boundaries”. In: *PRX Quantum* 6.4 (Nov. 12, 2025), p. 040330. DOI: 10.1103/qv65-vmzr. URL: <https://link.aps.org/doi/10.1103/qv65-vmzr> (visited on 11/16/2025).
- [28] Theodore J. Yoder, Eddie Schoute, Patrick Rall, Emily Pritchett, Jay M. Gambetta, Andrew W. Cross, Malcolm Carroll, and Michael E. Beverland. *Tour de Gross: A Modular Quantum Computer Based on Bivariate Bicycle Codes*. June 3, 2025. DOI: 10.48550/arXiv.2506.03094. arXiv: 2506.03094 [quant-ph]. URL: <http://arxiv.org/abs/2506.03094> (visited on 11/16/2025). Pre-published.
- [29] Daniel Litinski. “A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery”. In: *Quantum* 3 (Mar. 5, 2019), p. 128. DOI: 10.22331/q-2019-03-05-128. URL: <https://quantum-journal.org/papers/q-2019-03-05-128/> (visited on 11/25/2025).
- [30] Andrew J. Landahl and Ciaran Ryan-Anderson. *Quantum Computing by Color-Code Lattice Surgery*. July 18, 2014. DOI: 10.48550/arXiv.1407.5103. arXiv: 1407.5103 [quant-ph]. URL: <http://arxiv.org/abs/1407.5103> (visited on 11/25/2025). Pre-published.
- [31] Leonid P. Pryadko, Vadim A. Shabashov, and Valerii K. Kozin. “QDistRnd: A GAP Package for Computing the Distance of Quantum Error-Correcting Codes”. In: *Journal of Open Source Software* 7.71 (Mar. 22, 2022), p. 4120. ISSN: 2475-9066. DOI: 10.21105/joss.04120. arXiv: 2308.15140 [quant-ph]. URL: <http://arxiv.org/abs/2308.15140> (visited on 11/16/2025).
- [32] Stasiu Wolanski and Ben Barber. *Ambiguity Clustering: An Accurate and Efficient Decoder for qLDPC Codes*. Jan. 3, 2025. DOI: 10.48550/arXiv.2406.14527. arXiv: 2406.14527 [quant-ph]. URL: <http://arxiv.org/abs/2406.14527> (visited on 11/16/2025). Pre-published.

A Distance definitions

There are several definitions of distance relevant to our problem. The simplest is the code-capacity distance of the mid-cycle code. In the absence of dropouts, this is the weight of the lowest-weight logical operator of the code. When dropouts are introduced, we construct a subsystem code (Section 3.1). The relevant code-capacity distance is now the dressed distance, that is, the minimum weight of any operator

$$L \cdot G \tag{6}$$

where L acts only on the logical qubits, and G acts only on the gauge qubits, and we require that L is non-trivial. We can calculate the ‘dressed distance’ of the subsystem code with g gauge qubits by gauge-fixing. For each of the 2^g binary vectors z of length g , we construct a stabiliser code with an X parity check matrix given by the

parity check matrix of the code H^X with the rows of G^X corresponding to the 1s in z appended, and a Z parity check matrix constructed by appending the rows of G^Z where z is 0 to H^Z . We find the distance of each of these codes using the QDistRand package [31], and take the minimum.

We can also consider the distance of the end-cycle codes. In the dropout-free case, this is useful as the distance of the end-cycle codes gives an upper bound on the overall circuit distance. It is this code capacity distance of the end-cycle codes that we use to give the number of rounds R . However, it is not clear that this is a useful concept once we're dealing with subsystem codes: in general the end-cycle code is also a subsystem code, but the stabilisers of the end-cycle code may derive from gauge operators of the mid-cycle code, if we have an anticommuting quasi-stabiliser in the mid-cycle code all of whose anticommuting neighbours are being contracted in a given layer.

We can define the circuit-level distance in the usual way as the minimum number of errors that must occur in the circuit to flip the value of an observable whilst not triggering any detectors. This is very costly to estimate reliably, so in this work we focus on logical error rate instead, leaving a detailed examination of distance-reducing contraction layers to future work.

B Details of the CP-SAT problem

To enforce the quasi-stabiliser ordering constraints derived from the definition of product stabilisers (Section 3.1), we introduce FLAG, RESET, and COUNTER variables, each of which exists per-layer and has inter- and intra-layer constraints to model the scheduling we need.

- The FLAG variables, one for each constituent quasi-stabiliser, track whether each constituent quasi-stabiliser has been ‘measured yet’.
- In a given layer, if all the constituent FLAGS are true, we set the boolean RESET flag variable on the next layer to true. If RESET is true on a layer, we set FLAG variables back to false (unless they are immediately re-measured), and increment a COUNTER integer variable associated with the product.
- Then we require that all Z -type product stabilisers p_a^Z only start measuring once any X -type product stabilisers p_b^X that contain any quasi-stabilisers with which any of p_a^Z 's quasi-stabilisers anticommute wait until the COUNTER variables associated with the conflicting X -type stabilisers are all greater than their own COUNTER variables.
- Finally, we enforce that any ‘orphan’ quasi-stabilisers that do not form part of a product are also not measured at times they would interfere with the measurement of a product stabiliser.

Note that just because a quasi-stabiliser is not part of a product does not mean we can never form any detectors from it (Section 3.4), but that these detectors are likely to be quite limited in coverage.

Having built this problem, we optimise as follows.

- We require that all *stabilisers* are measured at least once over the L layers. Note that this includes both untouched stabilisers that did not suffer from defects rendering them anticommuting and product stabilisers formed from anticommuting quasi-stabilisers.
- We implicitly optimise over L as a primary objective by constructing CP-SAT problems of increasing L , starting at 2, and reconstructing with higher L if the solver deems the model INFEASIBLE. In our experiments, if L is required to be higher than 5, we record a failure.
- We explicitly optimise, within each L , to *maximise the minimum number of times* any stabiliser is measured. This is based on the idea that if, say, we need $L = 3$ layers to measure all the stabilisers but over those 3 layers we in fact measure each stabiliser twice, we may only need to wait $2d/3$ rounds between lattice surgery rounds to prevent harmful timelike errors. We leave examining this idea more closely to future work.
- Finally we tie-break between solutions by maximising the overall total number of quasi-stabilisers measured over all the layers.

The CP-SAT solver produces an assignment of a schedule (or none) for each quasi-stabiliser on each of the L layers, which fully specifies a single round of a syndrome extraction circuit.

C Memory simulations

A quantum memory experiment should, as far as possible, simulate a portion of what would, in an actual quantum computation, be a period of qubit idling preceded and followed by either more idling or logical operations. The Stim [21] memory experiments I ran to produce the plots in Appendix D have the following stages. For the noisy portion of the circuit, I assume $p = 0.001$ probability of depolarisation after one- and two-qubit gates, and include bit- and phase-flips in the appropriate basis after resets and before measurements also with probability p .

- First, we perform noiseless Stim Pauli-product measurements of all the quasi-stabilisers, to initialise the mid-cycle stabilisers of the code.
- Noiseless controlled- L^X and controlled- L^Z operations controlled on $2k$ noiseless reference qubits are performed to initialise X and Z observables. Concretely, for each X logical, we perform CNOTs controlled on the reference qubit and targeting each qubit in the support of the logical operator. For the Z logicals, the CNOTs are reversed.
- There are now R rounds of noisy syndrome extraction. R is set to the expected code-capacity distance of the end-cycle codes in the absence of dropouts (Appendix A), and it is this R that we use to calculate ‘per-round’ logical error rate. Each syndrome extraction round contains L contraction layers, each of which consists of t timesteps of simultaneous CNOTs, measurement and reset of the root qubits, and then the same t timesteps of CNOTs in reverse.
- The reference qubits are unentangled from the data qubits by noiselessly performing the same controlled-logical operations.
- Finally, we ‘close off’ detectors by noiselessly measuring all the stabilisers with Stim Pauli product measurement instructions.

D Performance plots

All memory experiments were simulated at $p = 0.001$ physical error rate and decoded with BP-AC [32].

Each of the following plots is divided into panels. In each panel I’ve fixed a number of qubits and couplers to drop out, and then sampled 30 dropout patterns, choosing uniformly among the qubits and couplers. ACID is then applied, producing a syndrome extraction circuit with L layers. In the background of each panel is a light-blue histogram showing how many of the dropout patterns resulted in a given L . In the foreground, drawn on top of each histogram bar, is a red violin plot (essentially a smoothed histogram aligned vertically), showing the distribution of logical error rate per round (LER) for the dropout patterns, selected by L . Syndrome extraction circuits consisting of more layers generally have a higher LER.

In the surface code plots, I include a comparison to a version of the LUCI algorithm, which always produces a syndrome extraction circuit with $L = 4$ layers (Appendix E).

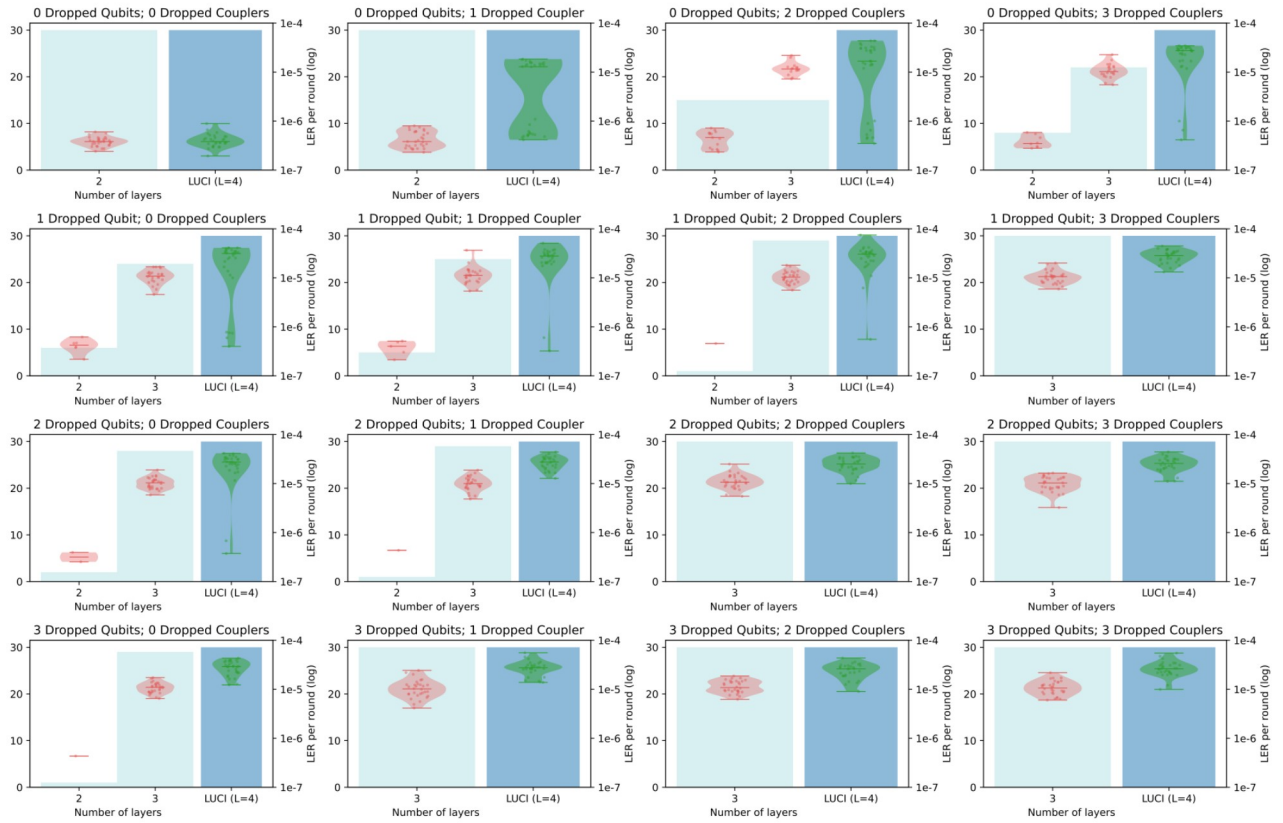
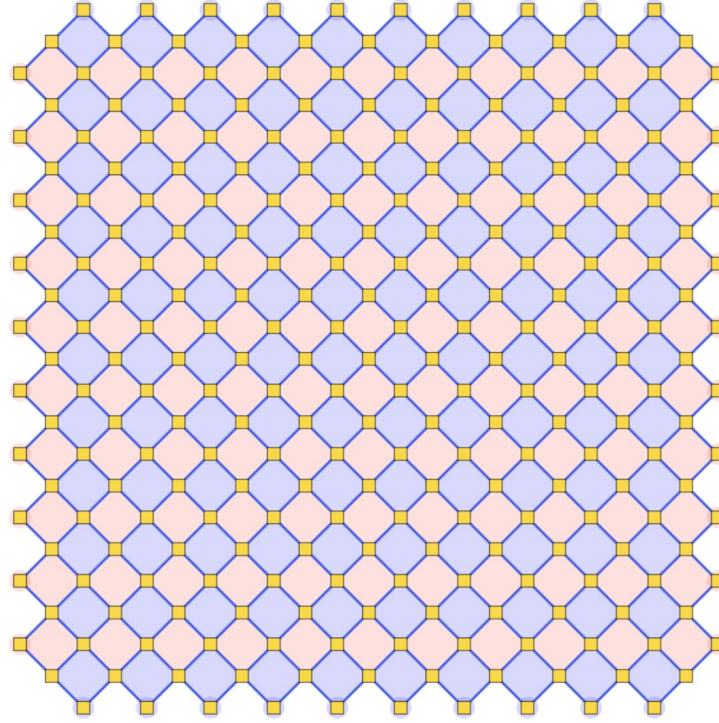
E Reproducing LUCI

To compare ACID against LUCI [9, 10] in the surface code, I implemented a version of the LUCI algorithm as a special case of ACID. The product stabilisers, gauge and logical operators are found by the usual ACID routine, as are the local contraction schedules and the constraints between them.

Rather than tasking the solver with minimising the number of contraction layers required for a syndrome extraction circuit, we divide the surface code quasi-stabilisers into four ‘colours’ 1-4 as described in the original LUCI paper, and require that the solver contract and measure the quasi-stabilisers of a given colour i on layer i . The solver is not required to use any particular schedule in measuring the required quasi-stabilisers, and is free to ‘fill in’ additional quasi-stabiliser contractions on each layer as far as possible. It is intended that this represents an upper bound on the flexibility of the LUCI algorithm. This implementation does not explicitly attempt to construct ‘shells’ as described in [10]. However, it does attempt to measure quasi-stabilisers that form part of product stabilisers as much as possible, including ‘redundant’ contractions that are not strictly required to measure product stabilisers, and it automatically assigns additional detectors in these cases, which likely amounts to the same thing.

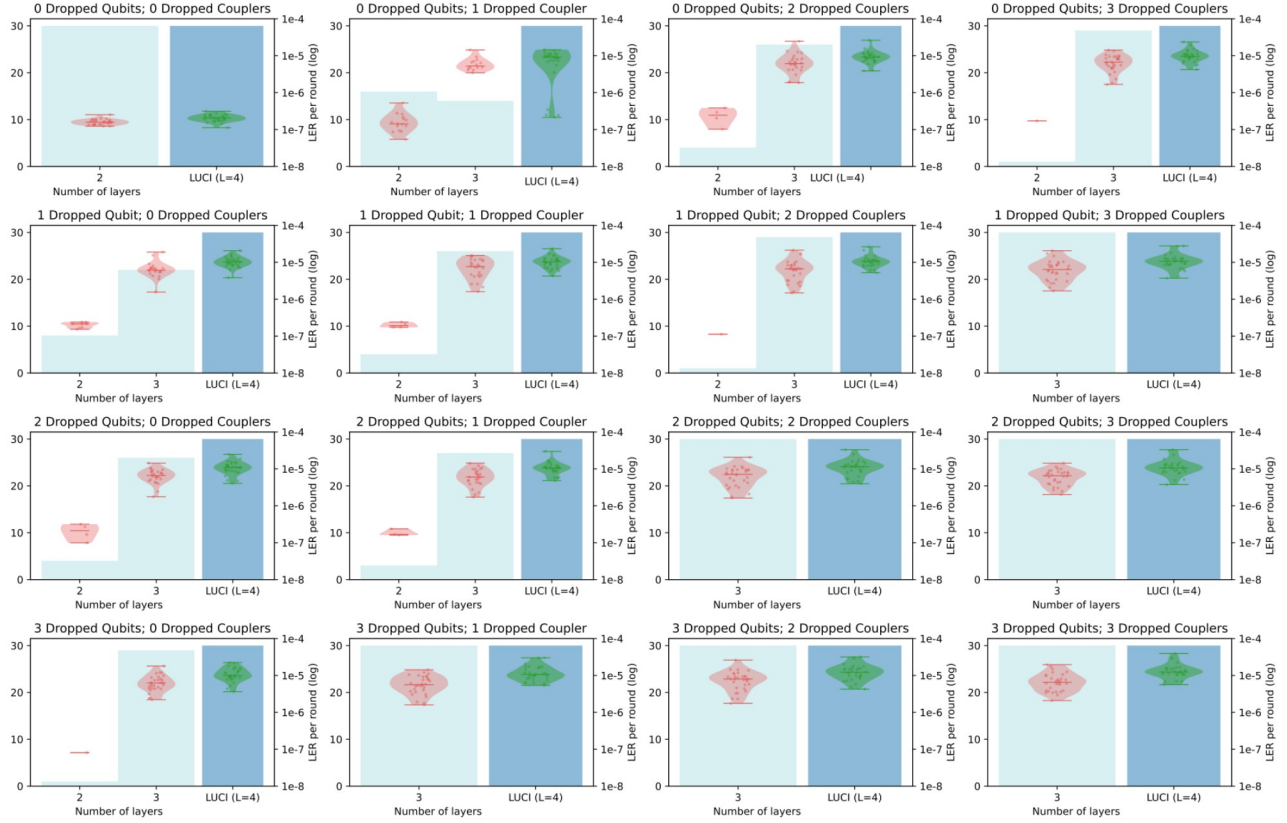
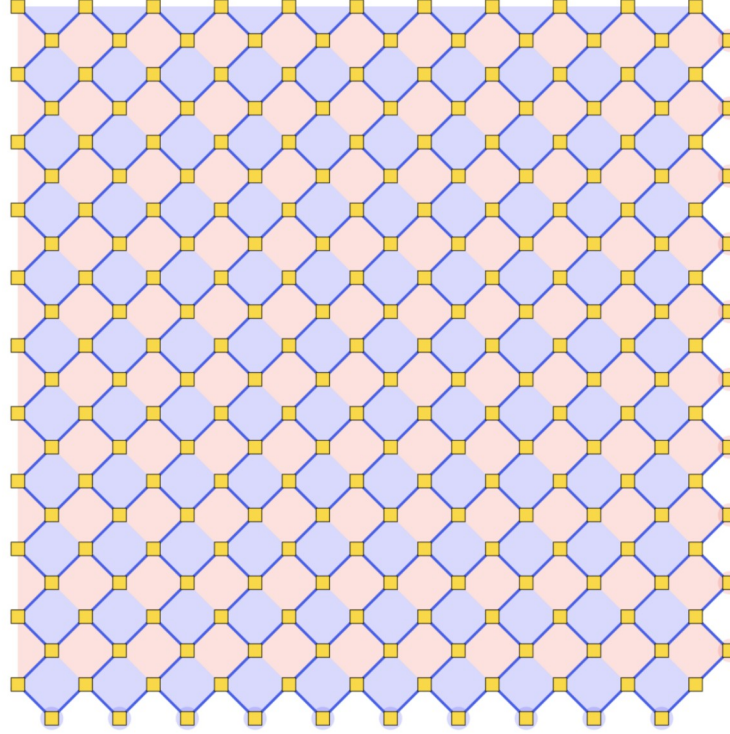
Distance-11 Square-Grid Surface Code

261 Qubits, 480 Couplers

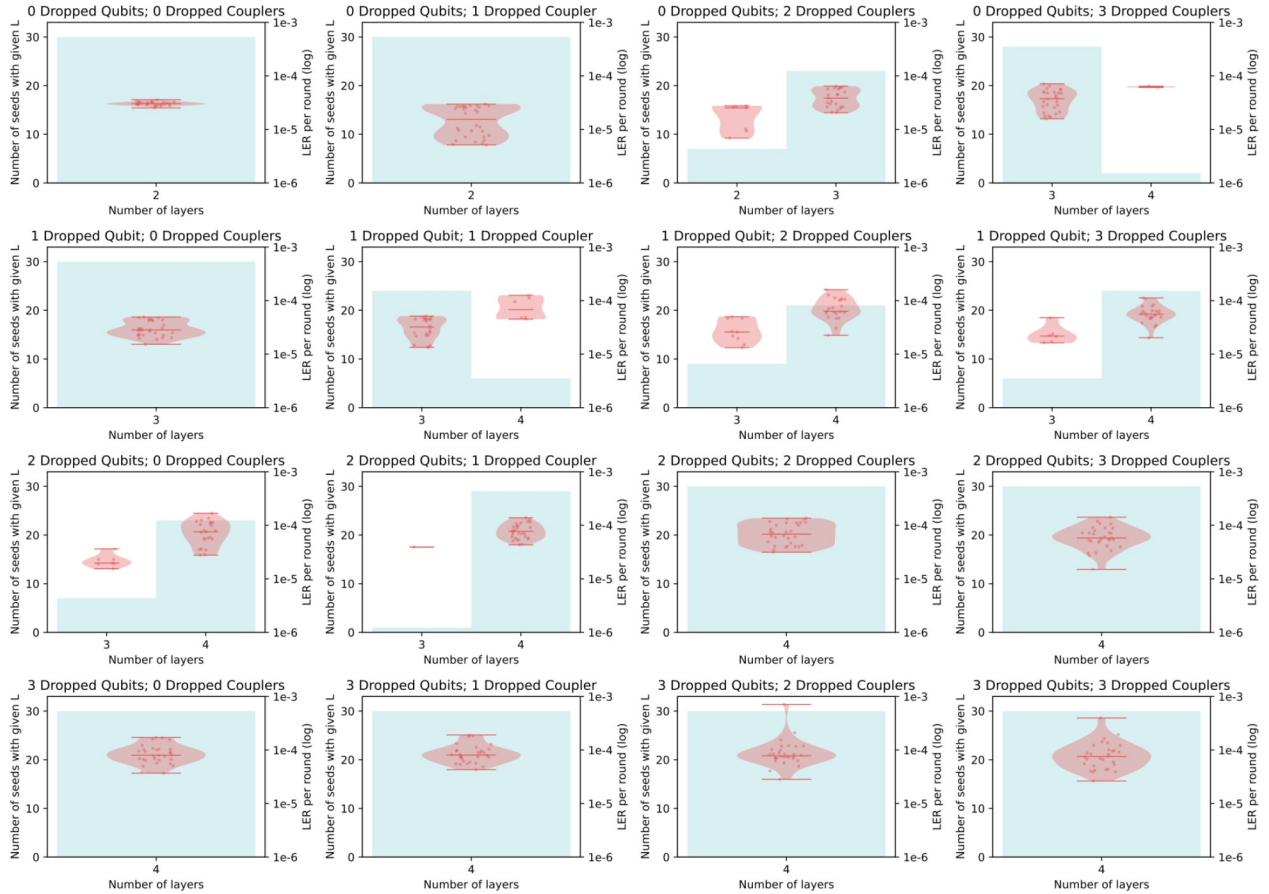
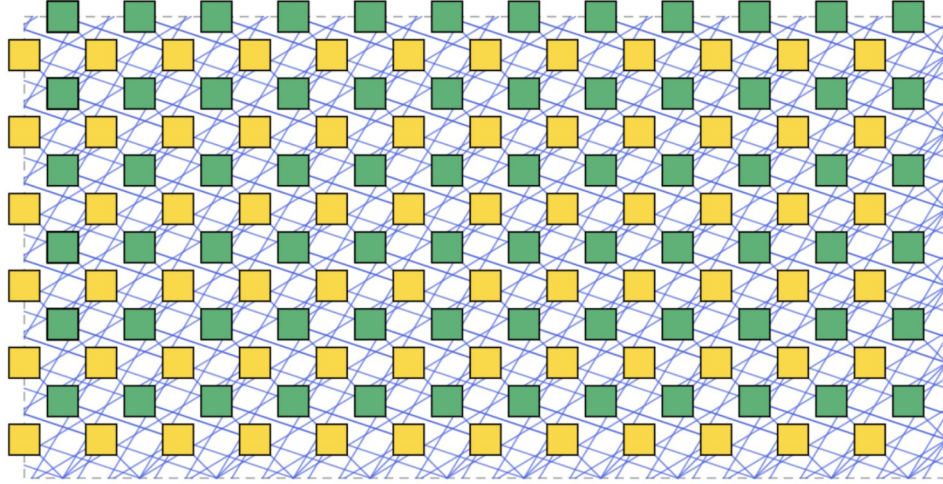


Distance-11 Hex-Grid Surface Code

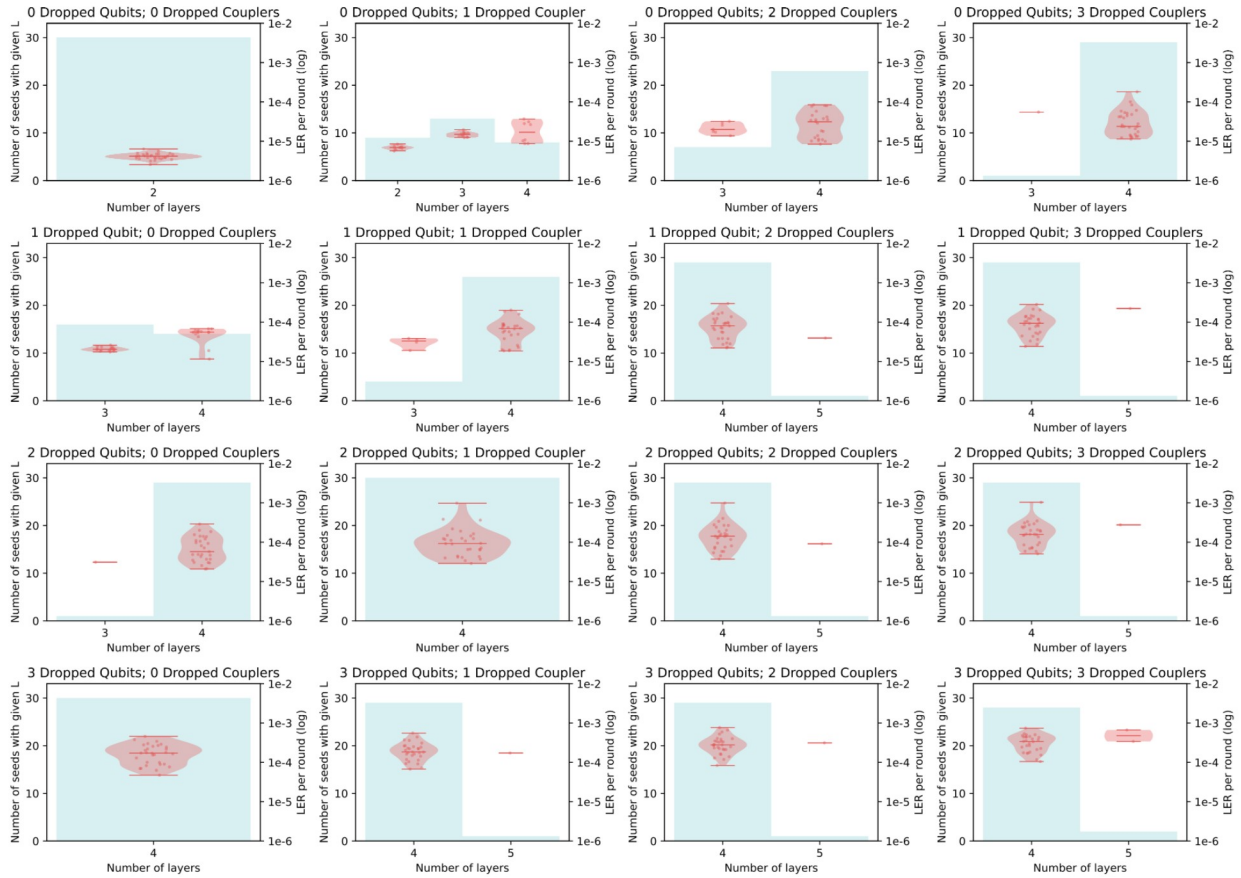
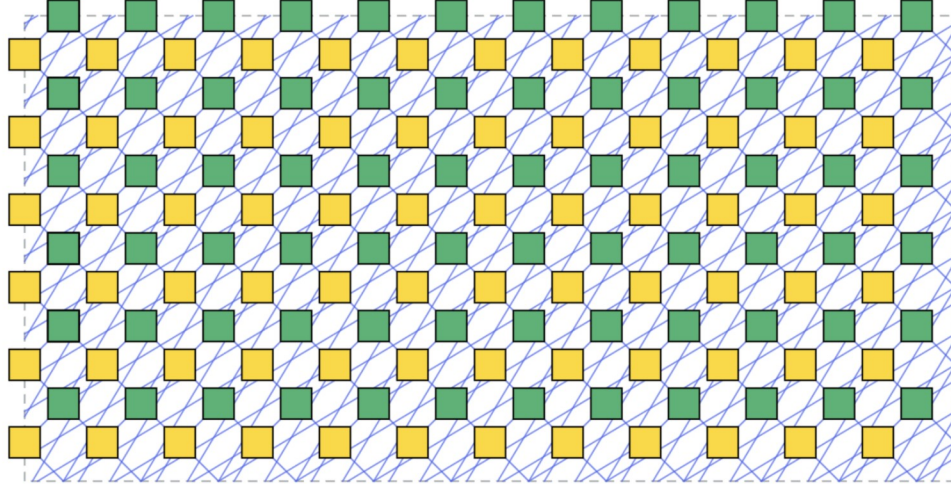
241 Qubits, 340 Couplers



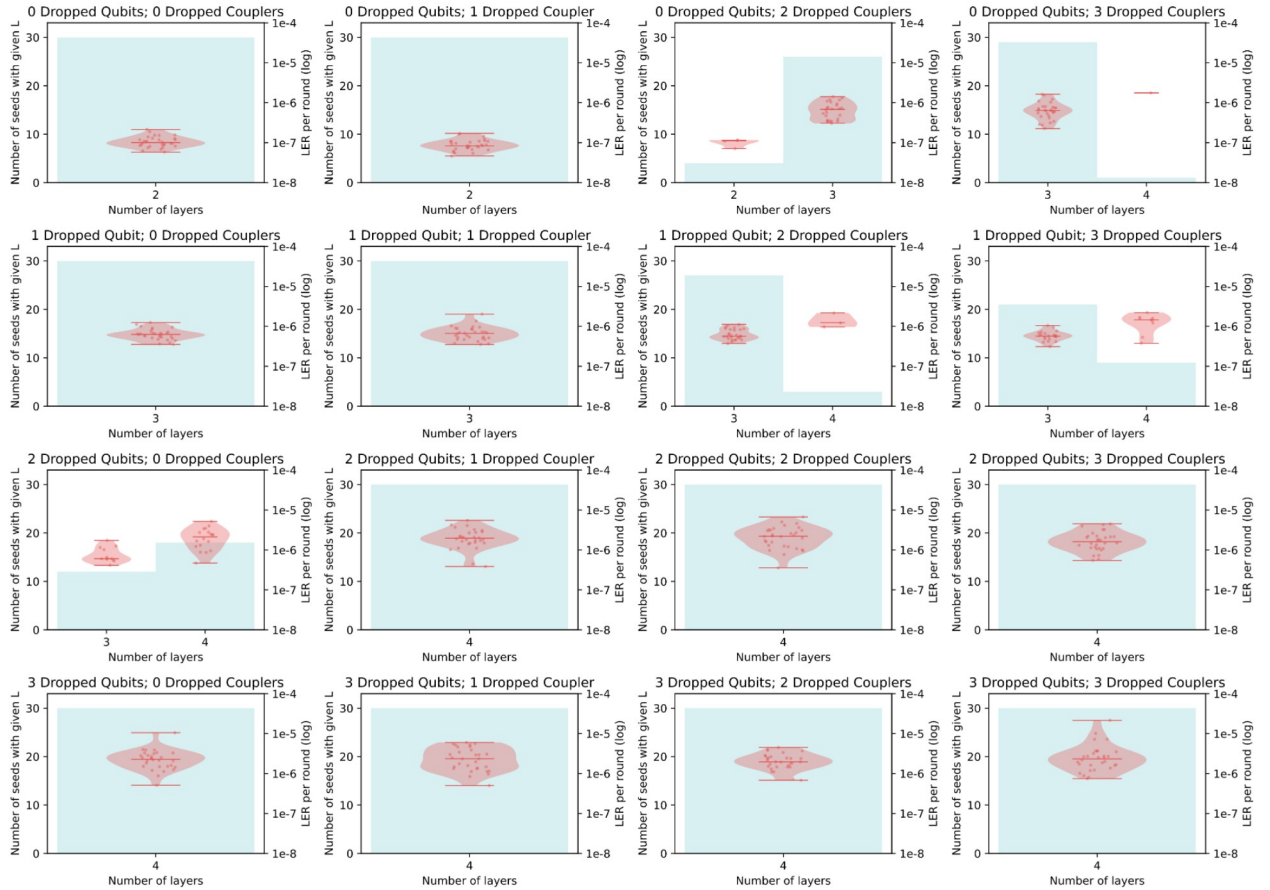
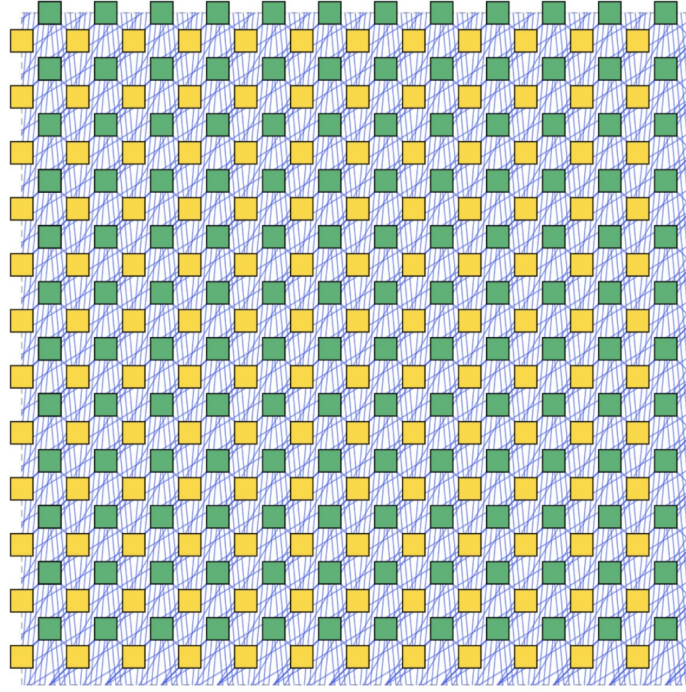
144-Qubit Bivariate Bicycle Code with Hexagonal Connectivity, $k=12$, Dropout-Free $d=6$
 144 Qubits, 432 Couplers



144-Qubit Bivariate Bicycle Code with Degree-5 Connectivity, $k=12$, Dropout-Free $d=6$
 144 Qubits, 360 Couplers

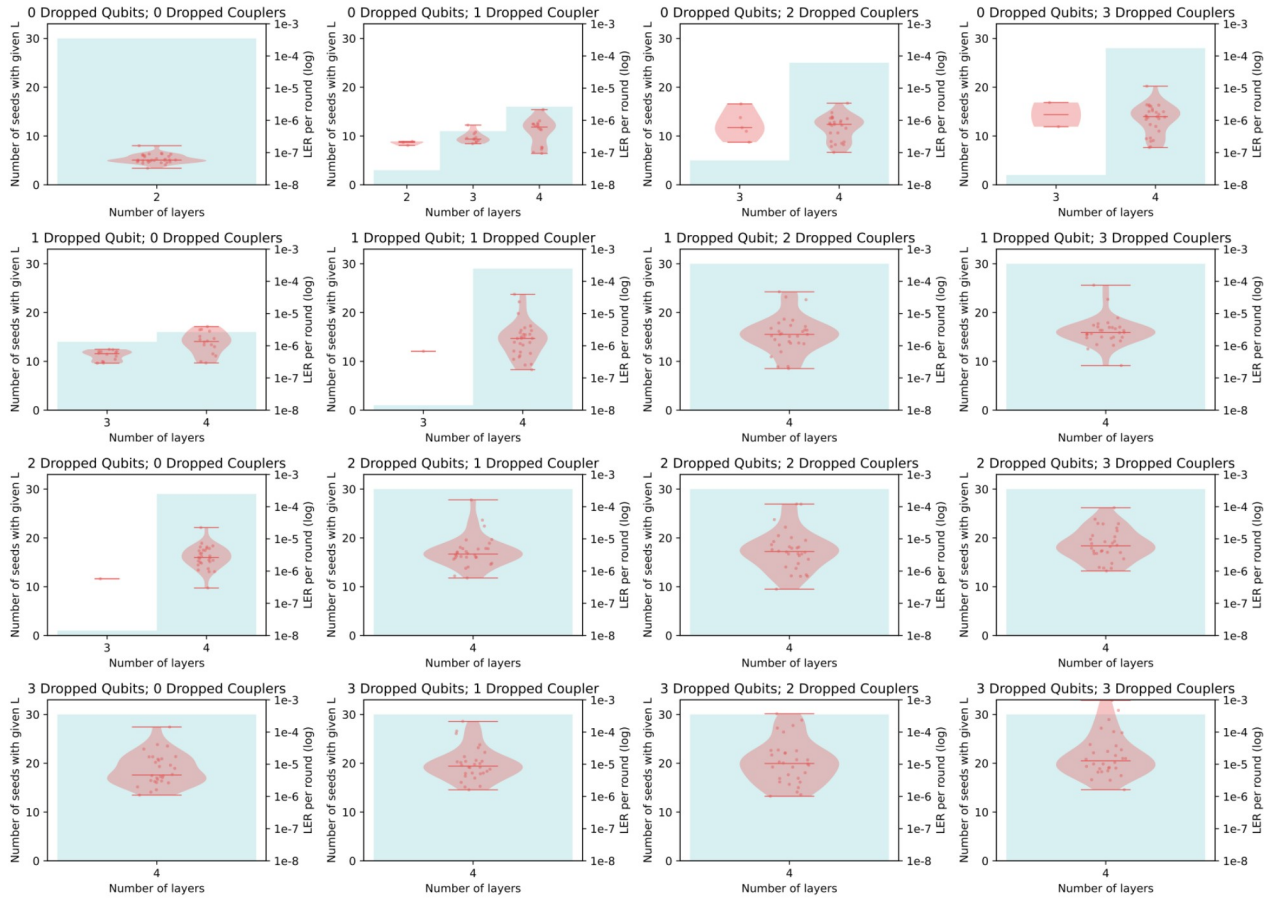
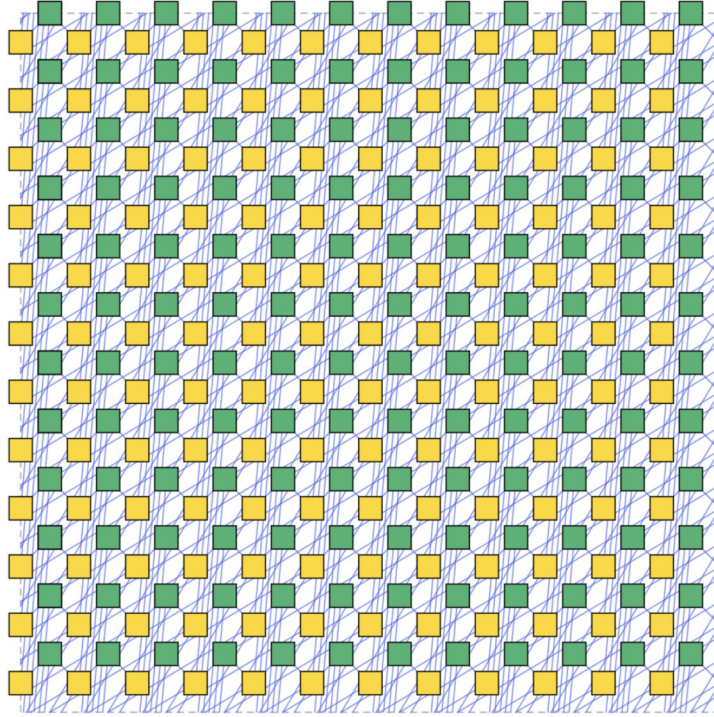


288-Qubit Bivariate Bicycle Code with Hexagonal Connectivity, $k=12$, Dropout-Free $d=12$
 288 Qubits, 864 Couplers

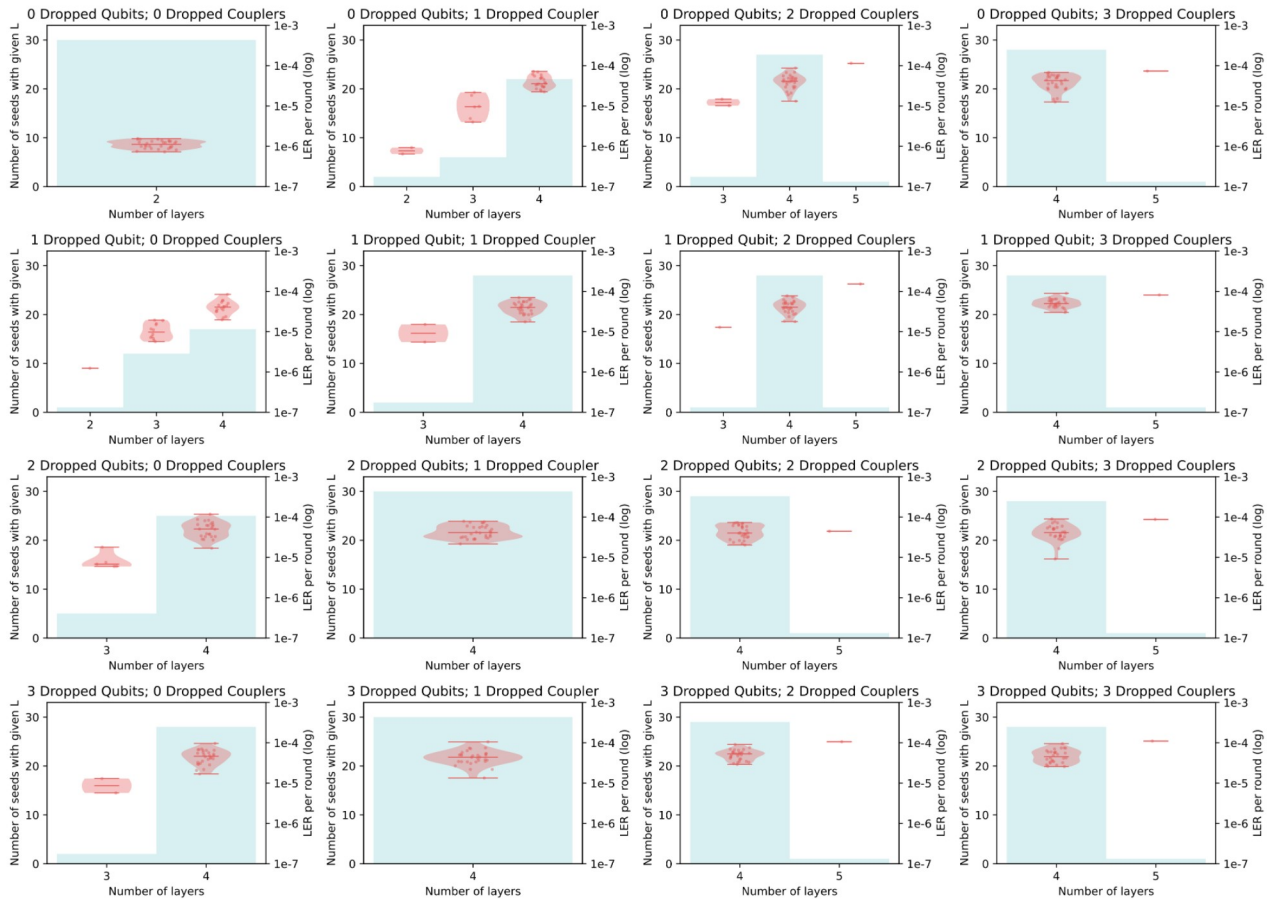
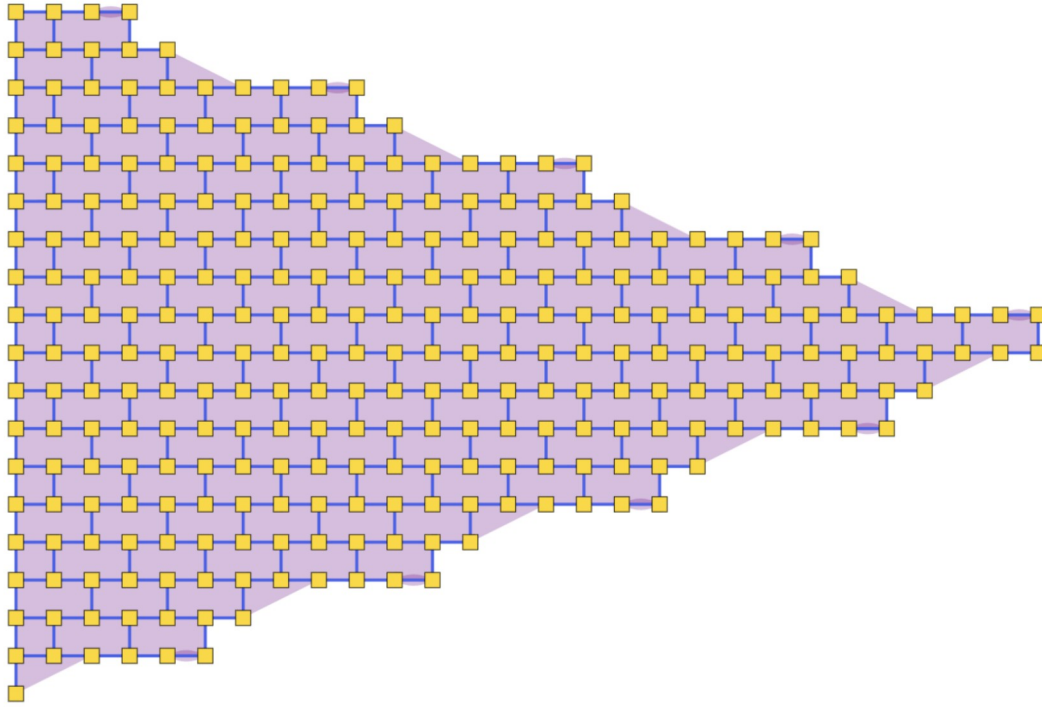


288-Qubit Bivariate Bicycle Code with Degree-5 connectivity, $k=12$, $d=12$

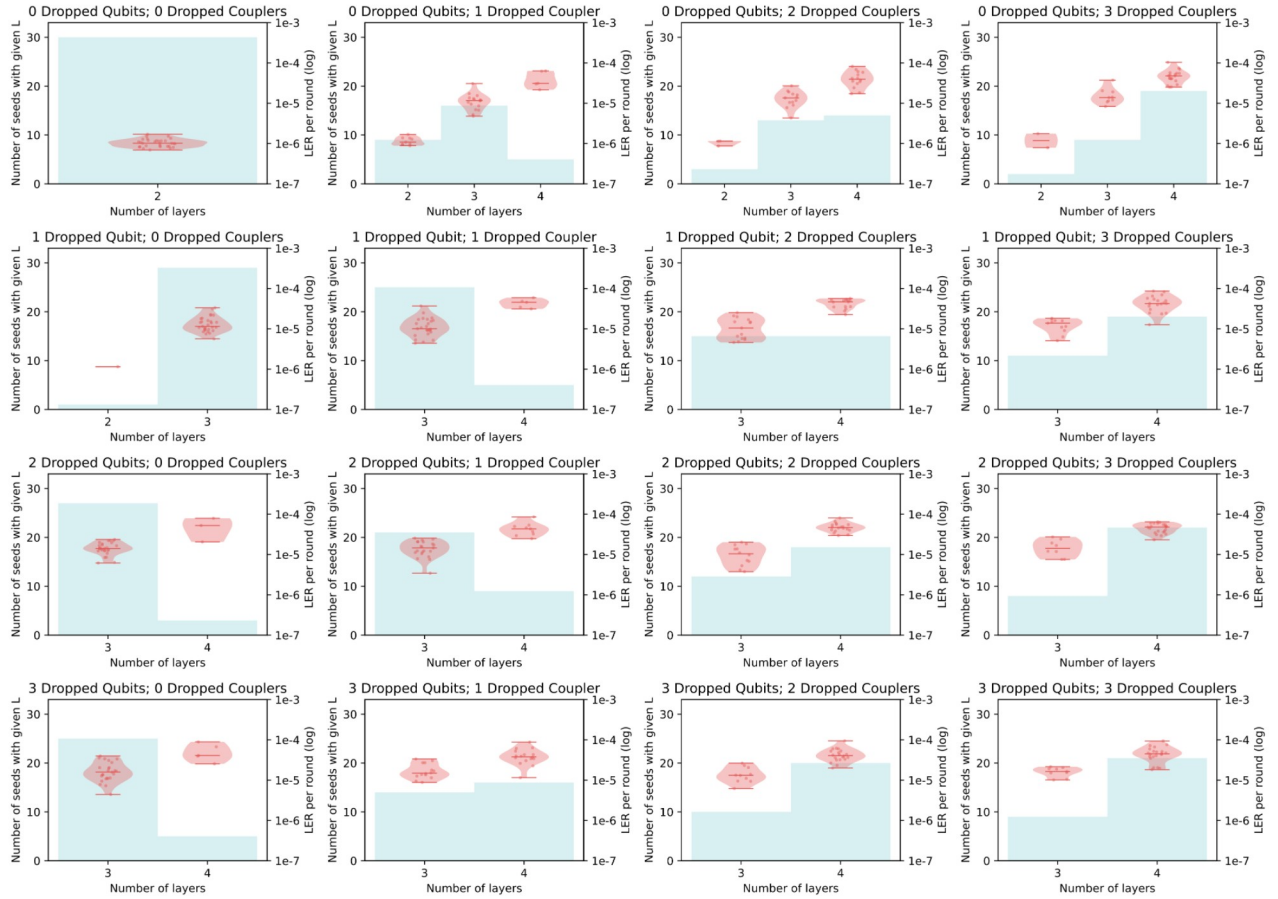
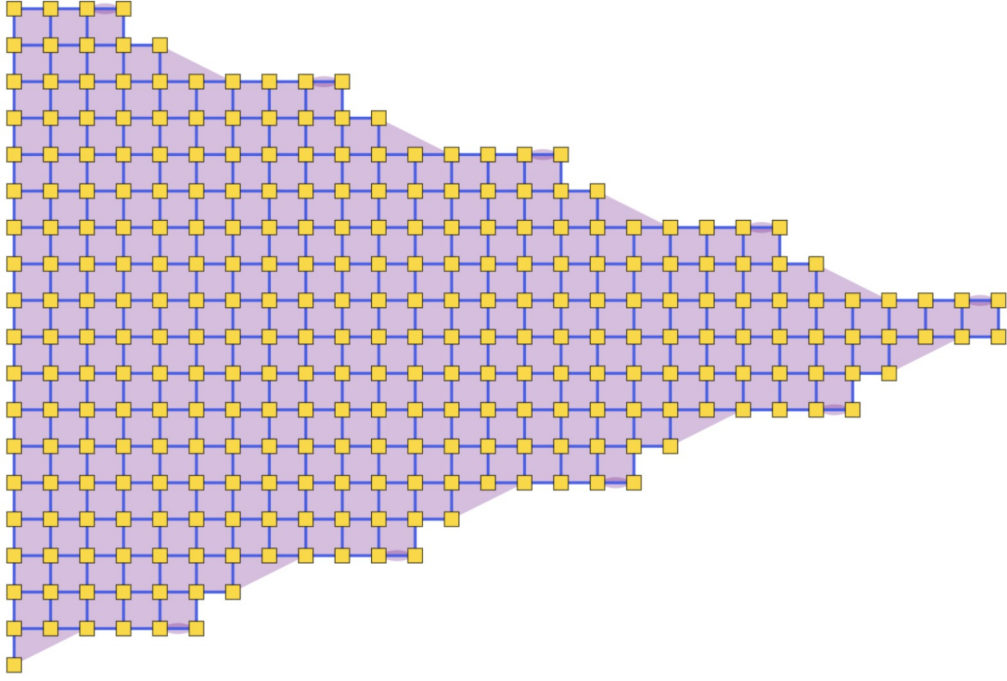
288 Qubits, 720 Couplers



Ancilla-free/Middle-out Colour Code with Hex Grid Connectivity, Dropout-Free $d=11$
289 Qubits, 414 Couplers



Ancilla-free/Middle-out Colour Code with Square Grid Connectivity, Dropout-Free $d=11$
289 Qubits, 531 Couplers



'Superdense-like' Colour Code with Square Grid Connectivity Dropout-Free $d=11$
181 Qubits, 325 Couplers

